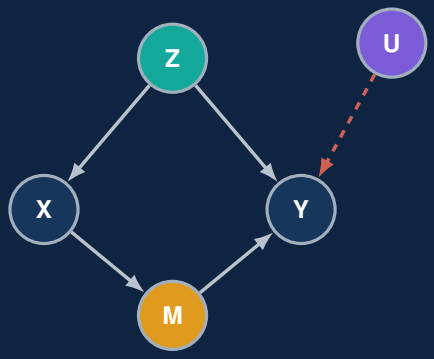


ADVANCED ARTIFICIAL INTELLIGENCE

Probabilistic Graphical Models and Causal Inference

Session 6

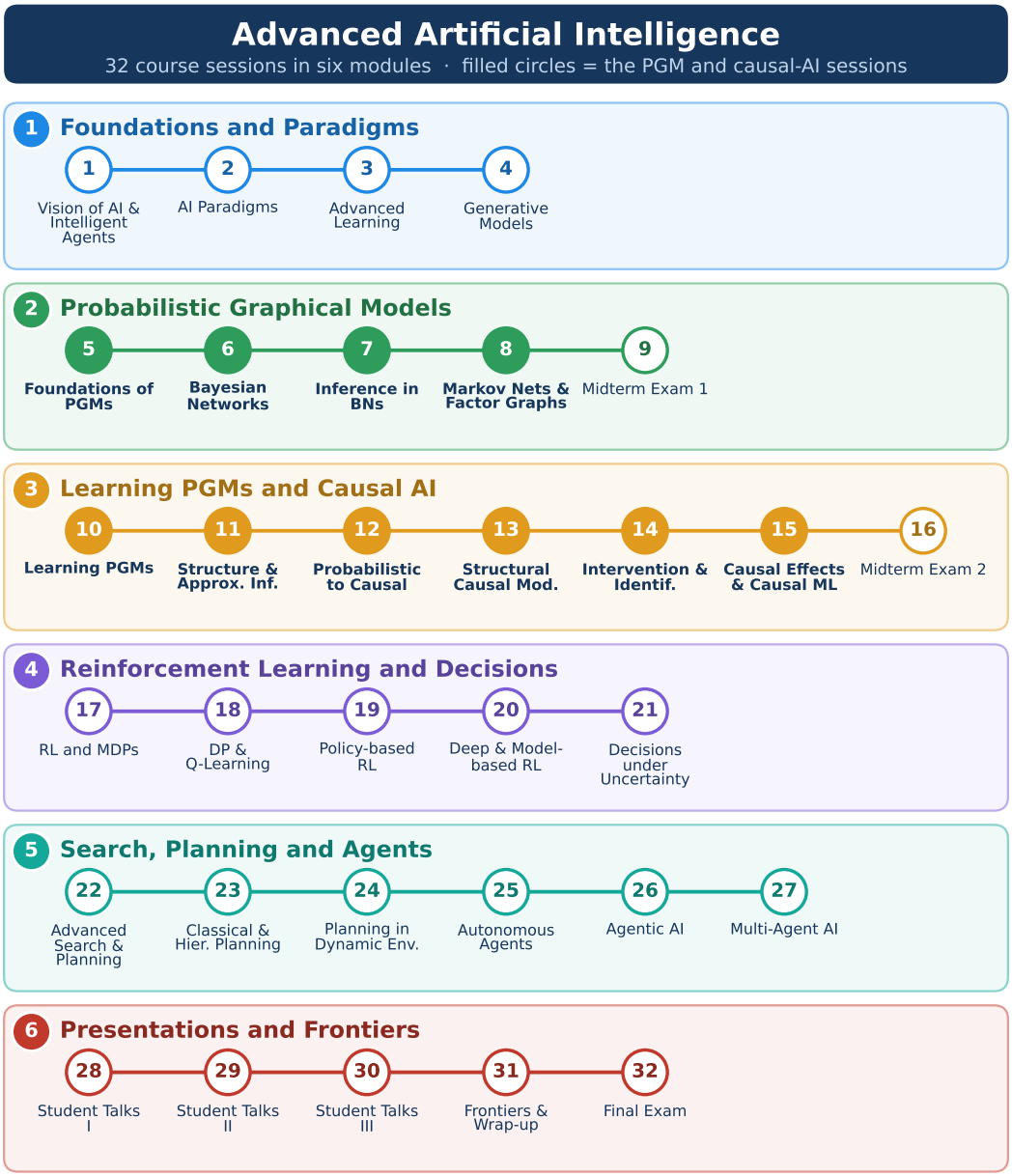
Bayesian Networks: DAGs and Independence Encoding

Lecture Notes

Instructor: Prof. M. Arashi

About these notes

These notes cover Session 6 of the course *Advanced Artificial Intelligence*, part of its probabilistic-graphical-model and causal-AI block (course sessions 5–15). The session follows a fixed structure: a roadmap and learning objectives, a "you are here" course map, the core theory organized in parts, a dedicated part on mathematical statistics, fully worked numerical and analytical examples, and a part connecting the topic to intelligent agents, agentic AI, reinforcement learning, multi-agent systems and the other sessions of the course. Every number is reproduced by the companion Python scripts (`python/s06_examples.py` for the numbers, `python/s06_figures.py` for the figures), and `questions/s06_questions.tex` contains ten questions with complete answers.



The Advanced Artificial Intelligence course. Filled circles mark the PGM and causal-AI sessions.

Contents

Session 6 Bayesian Networks: DAGs and Independence Encoding **1**

6.1 From Chain Rule to Graph . . . 2
6.2 Compact Conditional Probability Distributions . . . 5
6.3 Reasoning Patterns in a Bayesian Network . . . 6
6.4 Local Markov Property and Factorization . . . 8
6.5 d-Separation . . . 8
6.6 The Markov Blanket . . . 12
6.7 I-Maps, Minimal I-Maps and Perfect Maps . . . 13
6.8 Markov Equivalence . . . 15
6.9 Linear-Gaussian Bayesian Networks . . . 16
6.10 Likelihood, Parameters and Tests . . . 19
6.11 Worked Examples . . . 20
6.12 Bayesian Networks as World Models for Agents . . . 23
6.13 Summary . . . 27
6.14 Appendix: Proofs . . . 28

SESSION 6 ADVANCED ARTIFICIAL INTELLIGENCE

Bayesian Networks: DAGs and Independence Encoding

DAG structure, joint distributions, conditional probability tables, d-separation and conditional independence

Advanced Artificial Intelligence · Probabilistic Graphical Models and Causal Inference

Instructor: Prof. M. Arashi

PART I
Representation
DAG + CPTs = joint distribution; compact CPDs; reasoning patterns.

PART II
Independence
Local Markov, d-separation, Bayes ball, Markov blanket.

PART III
Maps & Equivalence
I-maps, perfect maps, faithfulness, Markov equivalence.

PART IV
Mathematical Statistics
Linear-Gaussian BNs, likelihood, CPT estimation, CI tests.

PART V
AI Connections
World models, tool-using agents, DBNs for RL, multi-agent BNs.

LEARNING OBJECTIVES

- Define a Bayesian network, write its chain-rule factorization and compute joint and conditional probabilities from its CPTs.
- Count free parameters and choose compact CPDs (deterministic, noisy-OR, tree, logistic, linear-Gaussian) when tables explode.
- Decide any query $X \perp\!\!\!\perp_d Y \mid Z$ by tracing chains, forks and colliders, and implement the linear-time Bayes-ball algorithm.
- Prove that factorization $\Leftrightarrow$ local Markov property; derive the Markov blanket; distinguish I-maps, minimal I-maps and perfect maps, and test Markov equivalence.
- Derive the covariance of a linear-Gaussian BN, the factorized likelihood and the count-based MLE of a CPT.
- Explain how BNs serve as world models for agents, as transition models in RL, and as distributed models in multi-agent systems.

Where we are. Session 5 gave us probability, conditional independence (CI) and the observation that a joint distribution is *compact* exactly when it factorizes. This session turns that observation into a modelling language, the *Bayesian network*. Sessions 7, 8 and 10 (course sessions 7, 8, 10) will compute with it, contrast it with undirected models and learn its CPTs; Sessions 12–15 reinterpret the same DAGs causally, and course sessions 17–27 reuse them as transition and world models.

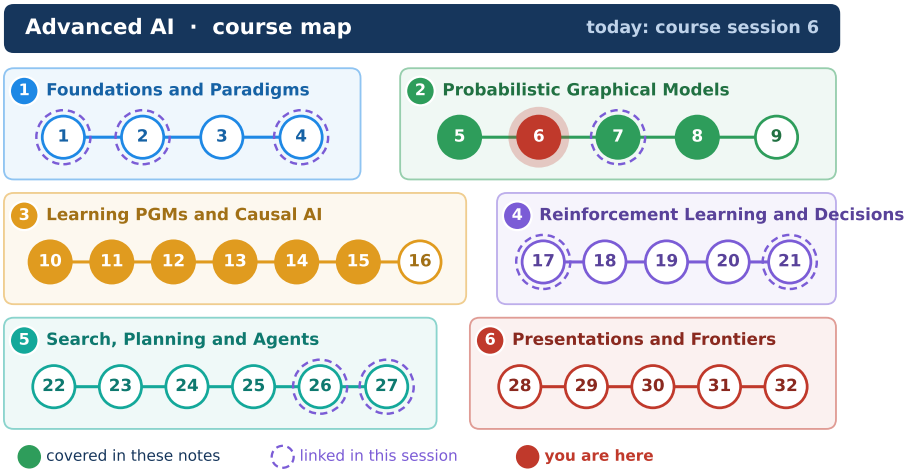


Figure 6.1. Where this session sits in the Advanced AI course. Course session 6 (ringed, "you are here") is the second session of the PGM block; dashed purple rings mark the course sessions this chapter links to explicitly.

OPENING MOTIVATION

Last session we established that conditional independence (CI) is what makes a joint distribution compact, and we practiced deriving CI algebraically from properties like symmetry and decomposition. That algebra works, but it does not scale: for a model with fifty variables, writing out and manually verifying every relevant CI statement by hand is unrealistic. What we need is a *representation* — something we can draw — from which every CI statement the model assumes can be read off by inspection. Today's answer is the Bayesian network: a directed acyclic graph (DAG) whose missing edges silently encode a whole family of conditional independencies, and a graphical test, d-separation, that lets us read them off at a glance.

Clinical decision support

Diseases point to symptoms and tests; the BN returns posteriors for any combination of findings.

Robotics and perception

Pose, map and sensor readings form a dynamic BN; each scan sharpens the belief over the hidden state.

LLM agents and tools

A troubleshooting agent keeps a small BN over causes and asks it which tool to call next.

PART I

Structure, Joint Distribution and CPTs

A graph for the qualitative assumptions, local tables for the numbers

PART I · REPRESENTATION

6.1 From Chain Rule to Graph

Recall the (always valid but generally useless) chain-rule factorization $P(x_1, \ldots, x_n) = \prod_i P(x_i \mid x_1, \ldots, x_{i-1})$. A Bayesian network is what you get when you are willing to *drop* some of the conditioning variables in each factor — keeping only a small, chosen subset $\mathrm{pa}(X_i)$ (the *parents* of

X_i) for each variable, and asserting that this is enough.

Notation pa(X): parents ch(X): children an(X): ancestors de(X): descendants nd(X): non-descendants MB(X): Markov blanket

A *directed acyclic graph* has no directed cycle, so its nodes admit at least one *topological order* in which every parent precedes its children. A *trail* (or path) is a sequence of nodes joined by edges of either orientation; a *directed path* follows the arrows.

Definition 6.1 (Bayesian network)**.** A Bayesian network is a pair (G, Θ) where G is a directed acyclic graph (DAG) with nodes $X_1, \dots, X_n$, and Θ specifies a conditional probability distribution $P(x_i \mid \mathrm{pa}(x_i))$ for every node. The network defines a joint distribution via the **chain rule for Bayesian networks**:

$$P(x_1, \dots, x_n) = \prod_{i=1}^{n} P\big(x_i \mid \mathrm{pa}(x_i)\big). \tag{6.1}$$

The right-hand side of (6.1) is automatically a probability distribution: summing out the variables in reverse topological order, each local CPD sums to one in turn. This factorization is a genuine assumption, called the **local Markov assumption**: each variable is conditionally independent of its non-descendants given its parents, $X_i \perp\!\!\!\perp \mathrm{nd}(X_i) \mid \mathrm{pa}(X_i)$. The DAG's edges are exactly the conditioning relationships the modeler chose to keep; every *missing* edge is an implicit claim of irrelevance. (Theorem 6.1 below proves that "factorizes" and "satisfies the local Markov assumption" are the same thing.)

Worked Example 6.1 (The student-grades network)**.** Consider five variables familiar from any classroom: course **D**ifficulty (easy/hard), student **I**ntelligence (low/high), the **G**rade earned, the student's **S**AT score, and the recommendation **L**etter quality. A modeler encodes these causal-looking assumptions: Grade depends on Difficulty and Intelligence; SAT depends only on Intelligence; the Letter depends only on the Grade. Figure 6.2 draws this. The chain rule for this network gives

$$P(d, i, g, s, l) = P(d)\,P(i)\,P(g \mid i, d)\,P(s \mid i)\,P(l \mid g),$$

five small factors with 15 free parameters in total (counted in (6.2) below) instead of one table over all $2 \times 2 \times 3 \times 2 \times 2 = 48$ joint configurations (G has three grade levels, the other variables two), which would need $48 - 1 = 47$ free numbers.

6.1.1 Conditional probability tables and a joint probability

A *conditional probability table* (CPT) lists, for every configuration u of the parents, a distribution over the child: each *row* sums to one. Figure 6.2 gives the full CPTs we use throughout the session. Values are coded as d^0=easy, d^1=hard; i^0=low, i^1=high; g^1=A, g^2=B, g^3=C; s^0=low, s^1=high SAT; l^0=weak, l^1=strong letter. For reference, the CPTs are $P(d^1) = 0.4$, $P(i^1) = 0.3$; $P(s^1 \mid i^0) = 0.05$, $P(s^1 \mid i^1) = 0.8$; $P(l^1 \mid g^1, g^2, g^3) = 0.9, 0.6, 0.01$; and $P(g^1, g^2, g^3 \mid i, d)$ equals $(0.3, 0.4, 0.3)$ for (i^0, d^0), $(0.05, 0.25, 0.7)$ for (i^0, d^1), $(0.9, 0.08, 0.02)$ for (i^1, d^0) and $(0.5, 0.3, 0.2)$ for (i^1, d^1).

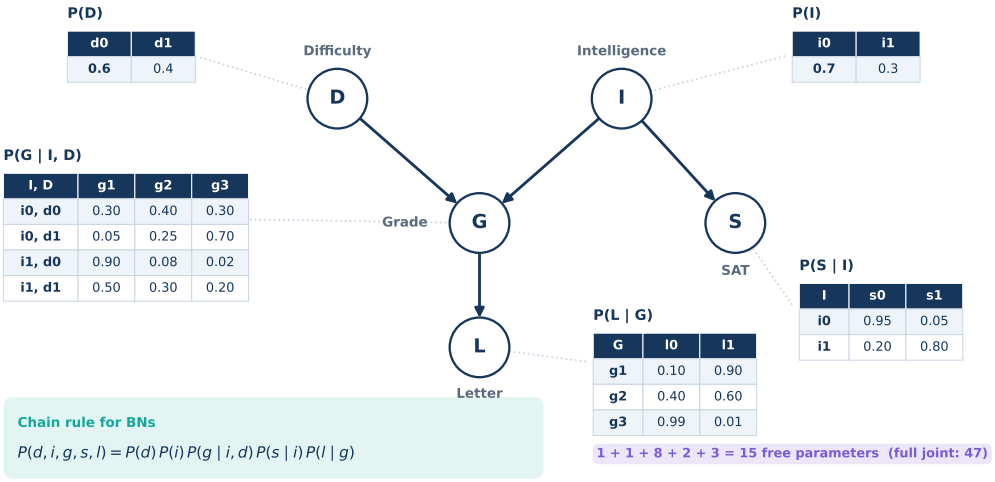


Figure 6.2. The student-grades Bayesian network with the conditional probability tables of Koller & Friedman (2009). Missing edges (e.g. no edge from D to S) are the model's independence assumptions; the five tables are all the numbers the model needs.

Worked Example 6.2 (One entry of the joint)**.** Compute the probability that the course is easy, the student is intelligent, gets a B, has a high SAT score and still receives a weak letter.
Solution. Read one entry from each CPT and multiply, as (6.1) instructs:

$$\begin{aligned} P(d^0, i^1, g^2, s^1, l^0) &= P(d^0)P(i^1)P(g^2 \mid i^1, d^0)P(s^1 \mid i^1)P(l^0 \mid g^2) \\ &= 0.6 \cdot 0.3 \cdot 0.08 \cdot 0.8 \cdot 0.4 = 0.18 \cdot 0.08 \cdot 0.8 \cdot 0.4 = 0.0144 \cdot 0.32 = 0.004608. \end{aligned}$$

Summing all 48 such products gives exactly 1 (each CPT row sums to one). Scanning all 48 products (done by the script below), the single most probable configuration is $(d^1, i^0, g^3, s^0, l^0)$, a hard course, a less intelligent student, a C and a weak letter, with probability $P(d^1)P(i^0)P(g^3 \mid i^0, d^1)P(s^0 \mid i^0)P(l^0 \mid g^3) = 0.4 \cdot 0.7 \cdot 0.70 \cdot 0.95 \cdot 0.99 = 0.184338 \approx 0.1843$.

PYTHON LAB · `s06_examples.py` · `ex_student_joint()`

```
P(d0,i1,g2,s1,l0) = 0.004608; sum over all 48 joint entries = 1.000000
free parameters:  BN = 15, full joint = 47; most probable configuration p = 0.1843
its factors P(d1)P(i0)P(g3|i0,d1)P(s0|i0)P(l0|g3) = 0.40*0.70*0.70*0.95*0.99 = 0.184338
```

Parameter counting. A CPT for a child with k_i values and parents with k_j values has $\prod_{j \in \mathrm{pa}(i)} k_j$ rows, each with $k_i - 1$ free numbers, so the network has

$$\dim(G) = \sum_{i=1}^{n} (k_i - 1) \prod_{j \in \mathrm{pa}(i)} k_j \tag{6.2}$$

free parameters. For the student network: $P(D)$ and $P(I)$ need 1 each, $P(G \mid I, D)$ has 4 rows of 2 free numbers (8), $P(S \mid I)$ needs 2 and $P(L \mid G)$ needs 3: $1 + 1 + 8 + 2 + 3 = 15$ in total, against $48 - 1 = 47$ for the unrestricted joint. With n binary variables and in-degree at most 2, each node contributes at most $(2-1) \cdot 2^2 = 4$, so $\dim(G) \le 4n$: linear rather than exponential. For example, if the first node has no parent, the second one parent and every other node two parents, $\dim(G) = 1{+}2{+}4(n{-}2) = 4n{-}5$; for $n = 30$ this is 115, against $2^{30} - 1 = 1{,}073{,}741{,}823 \approx 1.07 \times 10^9$ for the full joint (`s06_examples.py`, function `ex_param_scaling()`).

PART I · REPRESENTATION

6.2 Compact Conditional Probability Distributions

A tabular CPT with n binary parents has 2^n rows. Ten causes of a symptom already need 1024 numbers, few of which an expert can elicit or data can estimate. The remedy is a *structured* CPD: a parametric family for $P(x \mid u)$ with far fewer parameters.

Definition 6.2 (Noisy-OR CPD)**.** Let Y have binary parents $X_1, \dots, X_n$. Each active parent $(X_i = 1)$ independently succeeds in turning Y on with probability λ_i, and a *leak* turns Y on with probability λ_0 even when no parent is active. Then

$$P(Y = 0 \mid x_1, \dots, x_n) = (1 - \lambda_0) \prod_{i=1}^{n} (1 - \lambda_i)^{x_i}, \qquad P(Y = 1 \mid \mathbf{x}) = 1 - P(Y = 0 \mid \mathbf{x}). \quad (6.3)$$

The derivation is a small BN in itself: introduce hidden "activation" variables $A_i \sim \mathrm{Bern}(\lambda_i)$, independent, and let $Y = A_0 \vee \bigvee_i (X_i \wedge A_i)$. Then $Y = 0$ iff every active cause failed and the leak failed, which is the product (6.3). The model has $n + 1$ parameters instead of 2^n, and each has a direct meaning that an expert can provide.

Worked Example 6.3 (Noisy-OR: cough, cold, flu, allergy)**.** Illustrative numbers: leak $\lambda_0 = 0.05$, cold $\lambda_1 = 0.5$, flu $\lambda_2 = 0.7$, allergy $\lambda_3 = 0.3$; the three causes are independent with priors $\pi_1 = P(\text{cold}) = 0.20$, $\pi_2 = P(\text{flu}) = 0.05$, $\pi_3 = P(\text{allergy}) = 0.10$. (a) Find $P(\text{cough} \mid \text{cold}, \text{flu}, \text{no allergy})$ and $P(\text{cough} \mid \text{all three causes})$. (b) Does learning about the flu change our belief in a cold, given a cough?

Solution. (a) By (6.3), $P(\text{cough} \mid \mathbf{x} = (1, 1, 0)) = 1 - (1 - 0.05)(1 - 0.5)(1 - 0.7) = 1 - 0.95 \cdot 0.5 \cdot 0.3 = 1 - 0.1425 = 0.8575$; with all three causes, $1 - 0.1425 \cdot (1 - 0.3) = 1 - 0.09975 = 0.90025$. (b) Because the causes are independent, averaging (6.3) over them factorizes: $\mathbb{E}\big[(1 - \lambda_i)^{X_i}\big] = 1 - \pi_i \lambda_i$, so

$$\begin{aligned} P(\text{cough} = 0) &= (1 - \lambda_0) \textstyle\prod_i (1 - \pi_i \lambda_i) = 0.95 \cdot (1 - 0.2 \cdot 0.5)(1 - 0.05 \cdot 0.7)(1 - 0.1 \cdot 0.3) \\ &= 0.95 \cdot 0.9 \cdot 0.965 \cdot 0.97 = 0.80032, \qquad P(\text{cough}) = 0.19968 \approx 0.1997. \end{aligned}$$

Fixing cold $= 1$ replaces its factor 0.9 by $1 - \lambda_1 = 0.5$ and multiplies by π_1: $P(\text{cold}, \text{no cough}) = 0.2 \cdot 0.5 \cdot 0.95 \cdot 0.965 \cdot 0.97 = 0.088925$, so $P(\text{cold}, \text{cough}) = 0.2 - 0.088925 = 0.111075$ and $P(\text{cold} \mid \text{cough}) = 0.111075 / 0.19968 = 0.5563$. Fixing in addition flu $= 1$:

$$\begin{aligned} P(\text{cold}, \text{flu}, \text{cough}) &= 0.2 \cdot 0.05 \cdot \big[1 - 0.95 \cdot 0.5 \cdot 0.3 \cdot 0.97\big] = 0.01 \cdot 0.861775 = 0.00861775, \\ P(\text{flu}, \text{cough}) &= 0.05 \cdot \big[1 - 0.95 \cdot 0.3 \cdot 0.9 \cdot 0.97\big] = 0.05 \cdot 0.751195 = 0.03755975, \end{aligned}$$

so $P(\text{cold} \mid \text{cough}, \text{flu}) = 0.00861775 / 0.03755975 = 0.2294$. The flu *explains away* the cough, so the cold becomes much less likely ($0.5563 \to 0.2294$): noisy-OR parents form a collider, and the pattern of Part II appears numerically. Figure 6.3 (left) plots (6.3) as causes are added.

PYTHON LAB · s06_examples.py · ex_noisy_or()

```
x=(1, 1, 0):  P(cough=1|x) = 0.85750; x=(1, 1, 1):  P(cough=1|x) = 0.90025
P(cough=1) = 0.1997; P(cough=0) = 0.80032; P(cold=1, cough=1) = 0.111075
P(cold=1, flu=1, cough=1) = 0.00861775; P(flu=1, cough=1) = 0.03755975
P(cold=1 | cough) = 0.5563; P(cold=1 | cough, flu=1) = 0.2294
```

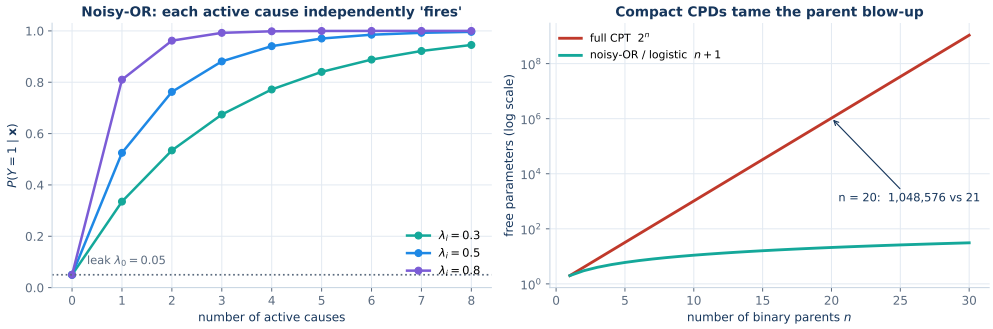


Figure 6.3. Left: under noisy-OR with leak $\lambda_0 = 0.05$ and k active causes of equal strength λ_i, $P(Y = 1 \mid \mathbf{x}) = 1 - 0.95(1 - \lambda_i)^k$: every extra active cause multiplies the probability that Y stays off by $1 - \lambda_i$. Right: free parameters of the CPD of a binary child with n binary parents; noisy-OR and logistic CPDs $(n + 1)$ grow linearly, tables (2^n) exponentially.

Other compact families (Table 6.1) follow the same idea: a *deterministic* CPD puts all mass on $y = f(u)$ (logical rules); a *tree-CPD* tests parents one at a time and stores distributions only at the leaves (context-specific independence); a *logistic CPD* sets $P(Y = 1 \mid \mathbf{u}) = \sigma(w_0 + \mathbf{w}^\top \mathbf{u})$. For continuous variables, the *linear-Gaussian CPD* is

$$X_i \mid \mathrm{pa}(X_i) = \mathbf{u} \;\sim\; \mathcal{N}\big(\beta_{i0} + \textstyle\sum_{j \in \mathrm{pa}(i)} b_{ij} u_j,\; d_i\big), \tag{6.4}$$

studied in depth in Part IV. Replacing the linear map by a neural network gives the *neural CPDs* of deep generative models (Part V).

Table 6.1. Compact CPDs for a child with n parents. "Params" counts free parameters for binary variables (or per child for continuous ones).

CPD	params	assumption	typical use
Tabular	2^n	none	small parent sets
Deterministic	0	$y = f(u)$ exactly	logic, bookkeeping variables
Noisy-OR	$n + 1$	causes act independently	diagnosis, failure analysis
Tree-CPD	#leaves	context-specific independence	expert systems, sparse rules
Logistic	$n + 1$	additive effect on log-odds	discriminative-style CPDs
Linear-Gaussian	$n + 2$	linear mean, constant variance	sensor fusion, Kalman models
Neural	weights	flexible mean / parameters	VAEs, autoregressive models

PART I · REPRESENTATION

6.3 Reasoning Patterns in a Bayesian Network

Before formalizing independence, watch how beliefs move through the student network (Fig. 6.4). Every number below is a ratio of sums of entries of the 48-entry joint (brute-force enumeration; efficient algorithms are Session 7's subject), and each can be checked by hand from the CPTs.

Three intermediate tables do most of the work, each obtained by summing out the parents of G:

$$P(G \mid i^0) = 0.6\,(0.3, 0.4, 0.3) + 0.4\,(0.05, 0.25, 0.7) = (0.20,\ 0.34,\ 0.46),$$
$$P(G \mid i^1) = 0.6\,(0.9, 0.08, 0.02) + 0.4\,(0.5, 0.3, 0.2) = (0.74,\ 0.168,\ 0.092),$$
$$P(G) = 0.7\,P(G \mid i^0) + 0.3\,P(G \mid i^1) = (0.362,\ 0.2884,\ 0.3496).$$

Causal reasoning (causes → effects). $P(l^1) = \sum_g P(g)P(l^1 \mid g) = 0.362 \cdot 0.9 + 0.2884 \cdot 0.6 + 0.3496 \cdot 0.01 = 0.502$; for a less intelligent student $P(l^1 \mid i^0) = 0.20 \cdot 0.9 + 0.34 \cdot 0.6 + 0.46 \cdot 0.01 = 0.389$; if the course is also easy, $P(l^1 \mid i^0, d^0) = 0.3 \cdot 0.9 + 0.4 \cdot 0.6 + 0.3 \cdot 0.01 = 0.513$.

Evidential reasoning (effects → causes). A C grade lowers intelligence from its prior 0.3 to $P(i^1 \mid g^3) = 0.3 \cdot 0.092/0.3496 = 0.079$. A weak letter alone: $P(l^0 \mid i^1) = 0.74 \cdot 0.1 + 0.168 \cdot 0.4 + 0.092 \cdot 0.99 = 0.23228$ and $P(l^0 \mid i^0) = 0.20 \cdot 0.1 + 0.34 \cdot 0.4 + 0.46 \cdot 0.99 = 0.6114$, so $P(i^1 \mid l^0) = \frac{0.3 \cdot 0.23228}{0.3 \cdot 0.23228 + 0.7 \cdot 0.6114} = \frac{0.069684}{0.497664} = 0.140$. Once the grade is known the letter adds nothing, $P(i^1 \mid l^0, g^3) = 0.079 = P(i^1 \mid g^3)$, because G blocks the chain $I \to G \to L$. Two effects of I can pull in opposite directions: combining the C with a high SAT score, $P(i^1 \mid g^3, s^1) = \frac{0.3 \cdot 0.092 \cdot 0.8}{0.3 \cdot 0.092 \cdot 0.8 + 0.7 \cdot 0.46 \cdot 0.05} = \frac{0.02208}{0.03818} = 0.578$.

Intercausal reasoning (between causes of a common effect). Given the C, learning that the course was hard raises intelligence from 0.079 to $P(i^1 \mid g^3, d^1) = \frac{0.3 \cdot 0.2}{0.3 \cdot 0.2 + 0.7 \cdot 0.7} = \frac{0.06}{0.55} = 0.109$: the difficulty partially *explains away* the poor grade. Symmetrically, given the C, a high SAT score (evidence for high intelligence) makes a hard course the more likely explanation: $P(d^1 \mid g^3) = 0.629$ rises to $P(d^1 \mid g^3, s^1) = 0.760$ (computed step by step in Worked Example 6.12). Given an A instead, a hard course makes intelligence *more* likely: $P(i^1 \mid g^1) = 0.3 \cdot 0.74/0.362 = 0.613$ but $P(i^1 \mid g^1, d^1) = \frac{0.3 \cdot 0.5}{0.3 \cdot 0.5 + 0.7 \cdot 0.05} = \frac{0.15}{0.185} = 0.811$.

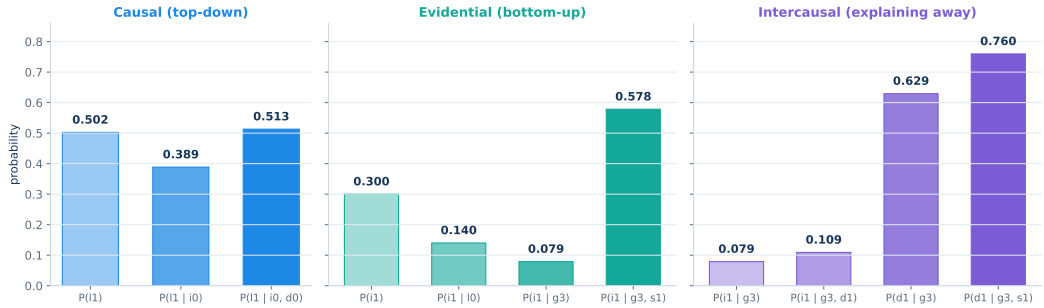


Figure 6.4. Three reasoning patterns in the student network, computed from the CPTs of Fig. 6.2 (the values derived in Section 6.3). Intercausal reasoning is the signature of a collider: D and I are marginally independent but become dependent once their common effect G is observed.

PYTHON LAB · s06_examples.py · ex_reasoning_patterns()

```
P(G) = [0.362, 0.2884, 0.3496]; P(G | i0) = [0.2, 0.34, 0.46]; P(G | i1) = [0.74, 0.168,
0.092]
P(l1)=0.502 P(l1|i0)=0.389 P(l1|i0,d0)=0.513 | P(i1|l0)=0.140 P(i1|g3)=0.079
P(i1|l0,g3)=0.079
P(i1|g3,s1)=0.578 | P(i1|g3,d1)=0.109 P(d1|g3)=0.629 P(d1|g3,s1)=0.760 P(i1|g1)=0.613
P(i1|g1,d1)=0.811
```

Influence flows along the graph in both directions; the only question is which paths are open, and that is exactly what d-separation answers.

PART II

Reading Independence off the Graph

Local Markov property, d-separation, Bayes ball and the Markov blanket

PART II · INDEPENDENCE

6.4 Local Markov Property and Factorization

Theorem 6.1 (Factorization ⇔ local Markov property)**.** *Let G be a DAG over $X_1, \dots, X_n$ and P a distribution over the same variables. Then P factorizes as in (6.1) if and only if P satisfies the local independencies*

$$\mathcal{I}_\ell(G) = \{\, X_i \perp\!\!\!\perp \mathrm{nd}(X_i) \setminus \mathrm{pa}(X_i) \mid \mathrm{pa}(X_i) \;:\; i = 1, \dots, n \,\}.$$

Proof. ($\Leftarrow$) Fix a topological order $X_1, \dots, X_n$. The general chain rule gives $P(\mathbf{x}) = \prod_i P(x_i \mid x_1, \dots, x_{i-1})$. In a topological order the predecessors of X_i are non-descendants of X_i and include $\mathrm{pa}(X_i)$. The local independence plus the decomposition property (Session 5) give $X_i \perp\!\!\!\perp \{X_1, \dots, X_{i-1}\} \setminus \mathrm{pa}(X_i) \mid \mathrm{pa}(X_i)$, so $P(x_i \mid x_1, \dots, x_{i-1}) = P(x_i \mid \mathrm{pa}(x_i))$.

($\Rightarrow$) Suppose (6.1) holds. Fix X_i; write $N = \mathrm{nd}(X_i)$ and $D = \mathrm{de}(X_i) \setminus \{X_i\}$. The set N is ancestral (a parent of a non-descendant is a non-descendant, otherwise the child would be a descendant), so there is a topological order listing N first, then X_i, then D. Sum the joint over D in reverse order: the last variable appears only in its own CPD, which sums to one; repeat. Hence $P(x_i, \mathbf{n}) = P(x_i \mid \mathrm{pa}(x_i)) \prod_{j \in N} P(x_j \mid \mathrm{pa}(x_j))$, and summing out x_i gives $P(\mathbf{n}) = \prod_{j \in N} P(x_j \mid \mathrm{pa}(x_j))$. Dividing, $P(x_i \mid \mathbf{n}) = P(x_i \mid \mathrm{pa}(x_i))$, which depends on $\mathbf{n}$ only through $\mathrm{pa}(x_i) \subseteq N$. That is $X_i \perp\!\!\!\perp N \setminus \mathrm{pa}(X_i) \mid \mathrm{pa}(X_i)$. □

The local Markov assumption directly gives us *some* independencies (each node from its non-descendants given its parents), but it does not obviously tell us about, say, $D \perp\!\!\!\perp S$ (are course difficulty and SAT score independent, unconditionally?). We need a general graphical procedure that answers *any* query of the form "is $X \perp\!\!\!\perp Y \mid Z$ implied by this graph?" The key is to understand how probabilistic influence flows along paths, one edge-triple at a time.

PART II · INDEPENDENCE

6.5 d-Separation

6.5.1 Three building blocks

Every path in a DAG is built from three possible local patterns at each intermediate node, shown in Fig. 6.5 and summarized in Table 6.2.

- **Chain** $X \to Z \to Y$: X influences Y only *through* Z. If we observe Z, that channel is cut: $X \perp\!\!\!\perp Y \mid Z$. If Z is unobserved, influence flows freely.

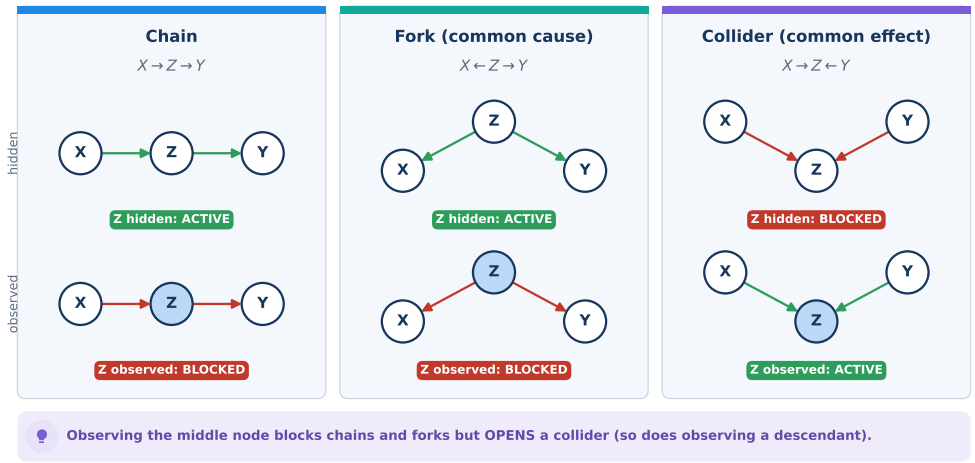


Figure 6.5. The three canonical connection patterns. Chains and forks transmit influence unless the middle node is observed; colliders do the opposite.

- **Fork** $X \leftarrow Z \rightarrow Y$: Z is a common cause; it creates a spurious correlation between X and Y. Again, conditioning on Z blocks the path: $X \perp\!\!\!\perp Y \mid Z$.
- **Collider** $X \rightarrow Z \leftarrow Y$: X and Y are marginally independent *causes* of a common effect Z — exactly like Session 5's XOR example, $Z = X \oplus Y$ with X, Y independent fair coins. (Session 5's enthusiast example is the opposite pattern, a fork: the enthusiast indicator is a common cause of the two ratings.) Here the logic flips: with Z unobserved, the path is blocked and $X \perp\!\!\!\perp Y$; but observing Z (or any descendant of Z) *opens* the path, inducing dependence ("explaining away").

Table 6.2. The three building blocks of d-separation.

Pattern	Graph	Z hidden	Z observed	Everyday reading
Chain	$X \rightarrow Z \rightarrow Y$	active	blocked	a mediator transmits influence
Fork	$X \leftarrow Z \rightarrow Y$	active	blocked	a common cause creates correlation
Collider	$X \rightarrow Z \leftarrow Y$	blocked	active	a common effect couples its causes

Definition 6.3 (d-separation)**.** A path between X and Y is *blocked* by a set of nodes Z if it contains a chain or fork node in Z, or a collider node such that neither the collider nor any of its descendants is in Z. Sets X and Y are *d-separated* by Z, written $X \perp\!\!\!\perp_d Y \mid Z$, if Z blocks *every* path between them; a path that is not blocked is *active*, and if some path is active, X and Y are *d-connected* given Z. We write $\mathcal{I}(G) = \{(X \perp\!\!\!\perp Y \mid Z) : X \perp\!\!\!\perp_d Y \mid Z\}$ for the set of all independencies the graph asserts.

"The collider or one of its descendants is in Z" is the same as "the collider is in $\mathrm{an}(Z) \cup Z$". d-connection means only that the graph does *not* assert independence; whether the variables are actually dependent depends on the CPT numbers (see faithfulness, Section 6.7.1). In the worked examples we write $X \not\perp\!\!\!\perp Y \mid Z$ for a d-connected pair in this sense: dependent for generic CPTs, and dependent for the CPTs at hand whenever a number is given.

Theorem 6.2 (Soundness of d-separation). *If $X \perp\!\!\!\perp_d Y \mid Z$ in DAG G, then $X \perp\!\!\!\perp Y \mid Z$ in* every *distribution P that factorizes according to G.*

The full proof is in the appendix (Section 6.14). It reduces the directed problem to an undirected one: keep only X, Y, Z and their ancestors, "marry" parents that share a child, drop the arrows, and show that (i) the distribution of the kept variables factorizes over this *moral ancestral graph*, (ii) d-separation in G implies ordinary graph separation there, and (iii) graph separation plus factorization implies CI by Session 5's factorized form of CI. A converse also holds — if X and Y are *not* d-separated given Z, then $X \not\perp\!\!\!\perp Y \mid Z$ for *almost every* choice of numerical parameters Θ consistent with G (Koller & Friedman 2009, Ch. 3; the property is called *faithfulness*, Section 6.7.1); we state this without proof, since only soundness (Theorem 6.2) is needed to trust that a graph never claims an independence that isn't really there.

INTUITION

d-separation is best learned by tracing paths with your finger, not by memorizing the definition. Ask, for each path between X and Y: "does this path pass through a fork or chain node that IS shaded (observed)?" $\rightarrow$ blocked. "Does it pass through a collider that is NOT shaded, and none of whose descendants are shaded?" $\rightarrow$ blocked. Otherwise the path is active. $X \perp\!\!\!\perp_d Y \mid Z$ requires *every* path to be blocked — one active path is enough to destroy independence.

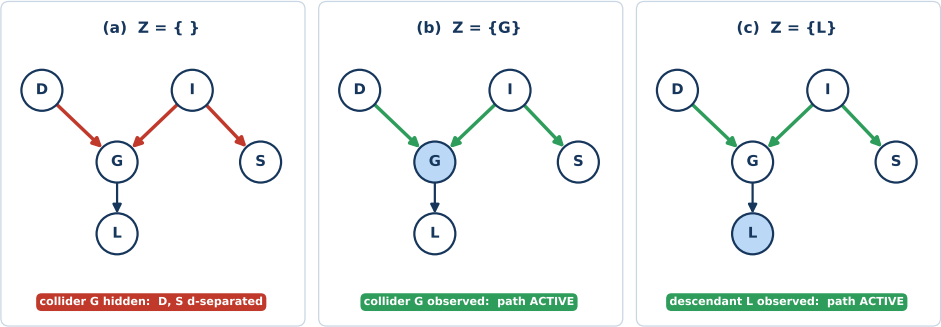


Figure 6.6. Observing the collider G (b), or its descendant L (c), opens the path $D \rightarrow G \leftarrow I \rightarrow S$ that is blocked when nothing is observed (a).

6.5.2 The Bayes-ball (reachability) algorithm

Checking every path is exponential in general, because a DAG can have exponentially many trails. The fix is to search over *(node, direction-of-arrival)* pairs instead of paths. A "ball" starts at X; when it arrives at Y from a child (travelling *up*) or from a parent (travelling *down*), the three building blocks decide where it may continue (Fig. 6.7). The first phase precomputes the set A of nodes that are in Z or have a descendant in Z; these are exactly the colliders that are active.

Proposition 6.1 (Linear complexity). *Reachable(G, X, Z) runs in time $O(|V| + |E|)$.*

Proof. Phase 1 is a backward graph search from Z: each node and edge is touched at most once. In phase 2 each of the $2|V|$ pairs (Y, d) is expanded at most once (the visited set), and expanding (Y, d) scans $\mathrm{pa}(Y) \cup \mathrm{ch}(Y)$, i.e. the edges incident to Y. Summing over pairs gives at most $2\sum_Y \deg(Y) = 4|E|$ insertions into L besides the initial one, hence at most $4|E| + 1$ pops, so

Algorithm 6.1 Reachable(G, X, Z): all nodes d-connected to X given Z (Koller & Friedman 2009, Ch. 3)

$A \leftarrow$ ancestors of Z (including Z) ▷ phase 1: active colliders
$L \leftarrow \{(X, \uparrow)\}$; $V \leftarrow \emptyset$; $R \leftarrow \emptyset$ ▷ to visit, visited, reachable
while $L \neq \emptyset$ **do**
  remove (Y, d) from L; **if** $(Y, d) \in V$ **continue**; add (Y, d) to V
  if $Y \notin Z$ **then** $R \leftarrow R \cup \{Y\}$
  if $d = \uparrow$ **and** $Y \notin Z$ **then** ▷ arrived from a child
    add $(P, \uparrow)$ for all $P \in \mathrm{pa}(Y)$ and $(C, \downarrow)$ for all $C \in \mathrm{ch}(Y)$ to L
  else if $d = \downarrow$ **then** ▷ arrived from a parent
    if $Y \notin Z$: add $(C, \downarrow)$ for all $C \in \mathrm{ch}(Y)$ ▷ chain continues
    if $Y \in A$: add $(P, \uparrow)$ for all $P \in \mathrm{pa}(Y)$ ▷ active collider
  end if
end while
return $R \setminus \{X\}$ ▷ $X \perp\!\!\!\perp_d Y \mid Z \iff Y \notin R$

the total work is $O(|V| + |E|)$. (Correctness: by induction on trail length, (Y, d) enters L iff some active trail from X arrives at Y in direction d; since the rule at a node depends only on the arrival direction, one visit per pair suffices.) □

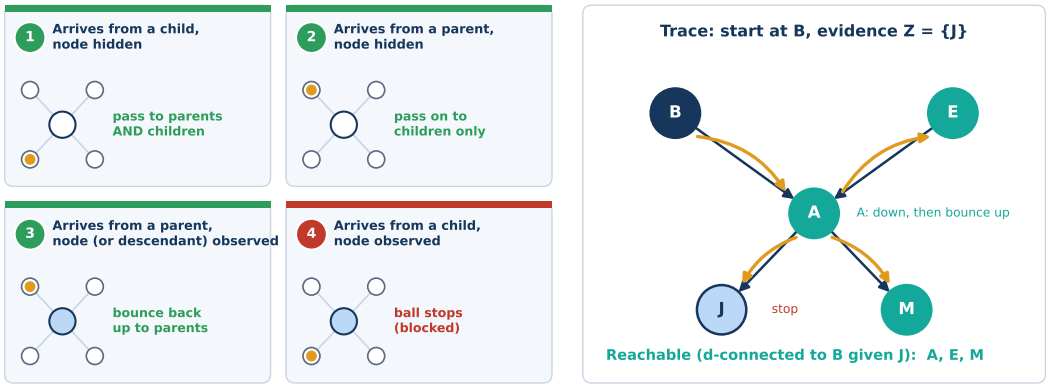


Figure 6.7. Bayes ball. Left: the four traversal rules (the amber dot marks where the ball arrives from). Right: a run on the alarm network from B with evidence J: the ball passes down through A, bounces up at A because its descendant J is observed, and reaches E — explaining away triggered by a descendant.

The script `ex_bayes_ball()` implements Algorithm 6.1 and tests it in three ways. (i) On the alarm network of Worked Example 6.15 ($B \to A \leftarrow E$, $A \to J$, $A \to M$) and on the student network it returns the reachable sets listed below. (ii) Simulation check: 200 random DAGs on 8 nodes (each forward pair $i < j$ joined with probability 0.3, a random evidence set of size 0–3 and a random start node, `numpy` seed 2); for every other non-evidence node the answer agrees with the d-separation test of `networkx` in all 1086 queries. (iii) Scaling: on the graphs with edges $i \to i+1$ and $i \to i+2$ (n nodes, $|E| = 2n - 3$) with evidence $Z = \{X_{n/2}\}$, the number of pops from L grows linearly, about $5n$, below the bound $4|E| + 1$ of the proof.

PYTHON LAB · s06_examples.py · ex_bayes_ball()

```
alarm from B: given [] -> ['A','J','M']; given ['A'] -> ['E']; given ['J'] -> ['A','E','M']
student from D: given [] -> ['G','L']; given ['G'] -> ['I','S']; given ['L'] ->
['G','I','S']
agreement with networkx on 200 random DAGs:  1086/1086
queue pops n=1000/4000/16000:  4990/19990/79990 (bound 4|E|+1 = 7989/31989/127989)
```

PART II · INDEPENDENCE

6.6 The Markov Blanket

Definition 6.4 (Markov blanket)**.** The Markov blanket of X in a DAG is $\mathrm{MB}(X) = \mathrm{pa}(X) \cup \mathrm{ch}(X) \cup \left(\bigcup_{C \in \mathrm{ch}(X)} \mathrm{pa}(C)\right) \setminus \{X\}$: parents, children, and the children's other parents (co-parents); see Fig. 6.8.

Theorem 6.3 (Markov blanket shields a node)**.** *Let $\mathcal{X} = \{X_1, \dots, X_n\}$ be the set of all variables and $\mathbf{x}_{-X}$ an assignment to all variables other than X. If P factorizes over G then for every node X, $X \perp\!\!\!\perp \mathcal{X} \setminus (\{X\} \cup \mathrm{MB}(X)) \mid \mathrm{MB}(X)$, and, wherever $P(\mathbf{x}_{-X}) > 0$,*

$$P(x \mid \mathbf{x}_{-X}) = \frac{P(x \mid \mathrm{pa}(x)) \prod_{C \in \mathrm{ch}(X)} P(c \mid \mathrm{pa}(c))}{\sum_{x'} P(x' \mid \mathrm{pa}(x)) \prod_{C \in \mathrm{ch}(X)} P(c \mid \mathrm{pa}(c))\big|_{x \to x'}}. \tag{6.5}$$

Proof. Write $P(x \mid \mathbf{x}_{-X}) = P(x, \mathbf{x}_{-X}) / \sum_{x'} P(x', \mathbf{x}_{-X})$ and substitute the factorization (6.1) in numerator and denominator. Every CPD that does not mention X — neither as child nor as parent — is a common factor and cancels. What remains is X's own CPD and the CPDs of its children, which is (6.5). These factors mention only X, $\mathrm{pa}(X)$, $\mathrm{ch}(X)$ and the parents of the children, i.e. X and $\mathrm{MB}(X)$. Hence the conditional depends on $\mathbf{x}_{-X}$ only through $\mathrm{MB}(X)$, which is the stated independence. □

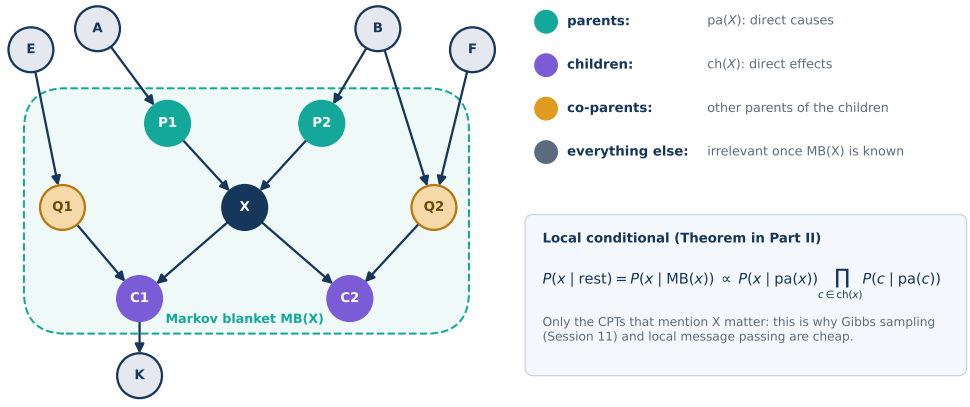


Figure 6.8. The Markov blanket of X (dashed region): parents, children and co-parents. Given these, the grey nodes are irrelevant to X. Co-parents are included because each child is a collider between X and its co-parent.

Worked Example 6.4 (A Markov blanket in the student network)**.** In the student network (Fig. 6.2), $\mathrm{MB}(G) = \{D, I, L\}$: parents D, I, child L, and no co-parents (L has no other parent);

MB$(I) = \{D, G, S\}$: children G, S and the co-parent D of G. Compute $P(G \mid d^1, i^1, s^1, l^1)$. *Solution.* By (6.5), $P(g \mid d^1, i^1, s^1, l^1) \propto P(g \mid i^1, d^1)\, P(l^1 \mid g)$. The unnormalized values for g^1, g^2, g^3 are $0.5 \cdot 0.9 = 0.45$, $0.3 \cdot 0.6 = 0.18$ and $0.2 \cdot 0.01 = 0.002$, with sum 0.632; dividing, $P(G \mid \cdot) = (0.45, 0.18, 0.002)/0.632 = (0.712, 0.285, 0.003)$. The SAT score is outside the blanket, and indeed dropping s^1 from the evidence leaves the answer unchanged.

PYTHON LAB · s06_examples.py · ex_markov_blanket()

```
MB(G) = ['D','I','L'] MB(I) = ['D','G','S']; unnormalised [0.45 0.18 0.002], sum = 0.632
P(G | d1,i1,s1,l1):  enumeration [0.712 0.285 0.003], local MB formula [0.712 0.285 0.003]
P(G | d1,i1,l1) (S dropped) [0.712 0.285 0.003]
```

PART III

I-Maps, Perfect Maps and Markov Equivalence

What exactly does a graph promise about a distribution?

6.7 I-Maps, Minimal I-Maps and Perfect Maps

Let $\mathcal{I}(P)$ be the set of all CI statements that hold in P. A graph is useful if what it asserts is true; it is *maximally* useful if it asserts everything that is true.

Definition 6.5 (I-map, minimal I-map, perfect map)**.** A DAG G is an *I-map* of P if $\mathcal{I}(G) \subseteq \mathcal{I}(P)$. It is a *minimal I-map* if removing any edge destroys this property. It is a *perfect map* (P-map) if $\mathcal{I}(G) = \mathcal{I}(P)$.

By Theorems 6.1 and 6.2, G is an I-map of P if and only if P factorizes over G. The complete DAG (every pair joined) asserts nothing and is therefore an I-map of every distribution — a correct but useless model. Minimal I-maps are constructed greedily: fix an order $X_1, \ldots, X_n$ and choose as pa(X_i) a smallest subset $U \subseteq \{X_1, \ldots, X_{i-1}\}$ with $X_i \perp\!\!\!\perp \{X_1, \ldots, X_{i-1}\} \setminus U \mid U$. By the ($\Leftarrow$) part of Theorem 6.1 the result is an I-map, and it is minimal by construction.

Worked Example 6.5 (The order of the variables matters)**.** Build minimal I-maps of the student distribution with three variable orders, assuming that the CI statements true in P are exactly those of the student DAG (faithfulness, Section 6.7.1), so that d-separation in Fig. 6.2 can serve as the CI oracle. Count parameters with (6.2) (G has 3 values, the others 2).
Solution. Order D, I, G, S, L ("causal"). D: no predecessors. I: $I \perp\!\!\!\perp D$, so no parent. G: no proper subset $U \subset \{D, I\}$ makes $G \perp\!\!\!\perp \{D, I\} \setminus U \mid U$, so pa$(G) = \{D, I\}$. S: $S \perp\!\!\!\perp \{D, G\} \mid I$, so pa$(S) = \{I\}$. L: $L \perp\!\!\!\perp \{D, I, S\} \mid G$, so pa$(L) = \{G\}$. This is the original 4-edge graph with $1 + 1 + 8 + 2 + 3 = 15$ parameters.
Order L, S, G, I, D ("diagnostic"). The same procedure gives the 7 edges $L \to S$, $L \to G$, $S \to G$, $S \to I$, $G \to I$, $G \to D$, $I \to D$ and $1 + 2 + (2 \cdot 4) + (1 \cdot 6) + (1 \cdot 6) = 23$ parameters (for L; $S \mid L$; $G \mid L, S$; $I \mid S, G$; $D \mid G, I$).
Order L, D, S, I, G. Given L (a descendant of the collider G) the variables D, S, I are all

mutually dependent, so the result has 9 of the 10 possible edges, $L \to D$, $L \to S$, $D \to S$, $L \to I$, $D \to I$, $S \to I$, $L \to G$, $D \to G$, $I \to G$ (only S–G is missing, since $G \perp\!\!\!\perp S \mid L, D, I$), and $1 + 2 + 4 + 8 + (2 \cdot 8) = 31$ parameters.
All three are I-maps of the same P; only the first is a perfect map (the other two have extra edges and therefore fail to assert some true CI statements, e.g. $D \perp\!\!\!\perp I$). Orders that follow causal direction tend to produce sparse graphs — an empirical observation that Sessions 12–13 will elevate to a principle.

PYTHON LAB · s06_examples.py · ex_minimal_imap()

```
order DIGSL: 4 edges, 15 params; order LSGID: 7 edges, 23 params; order LDSIG: 9 edges, 31
params
LSGID edges [(G,D),(G,I),(I,D),(L,G),(L,S),(S,G),(S,I)]
LDSIG edges [(D,G),(D,I),(D,S),(I,G),(L,D),(L,G),(L,I),(L,S),(S,I)]
```

Not every distribution has a perfect map. Take Session 5's XOR example: X, Y fair independent coins and $Z = X \oplus Y$. Every pair is independent, yet the three are jointly dependent. A DAG asserting $X \perp\!\!\!\perp Y$, $X \perp\!\!\!\perp Z$ and $Y \perp\!\!\!\perp Z$ must have no edges at all (adjacent nodes are never d-separated), and the empty graph asserts mutual independence, which is false. So no DAG is a P-map; the best we can do is a minimal I-map such as $X \to Z \leftarrow Y$. Session 8 shows distributions (the four-cycle) whose natural graph is undirected.

6.7.1 Faithfulness

Definition 6.6 (Faithfulness)**.** P is *faithful* to G if G is a perfect map of P, $\mathcal{I}(G) = \mathcal{I}(P)$: P factorizes over G, and every CI that holds in P is implied by d-separation in G.

Faithfulness fails only through exact numerical coincidences: the unfaithful CPT values form a Lebesgue-null set (Koller & Friedman 2009, Ch. 3); in linear-Gaussian models, the zero set of a polynomial in the edge weights.

Worked Example 6.6 (Path cancellation)**.** Let $X = \varepsilon_X$, $M = bX + \varepsilon_M$, $Y = aX + cM + \varepsilon_Y$ (edges $X \to M \to Y$ and $X \to Y$) with weights $b = 1.2$, $c = -0.5$, $a = 0.6$ and independent $\mathcal{N}(0, 1)$ noises. Show that X and Y are independent although $X \to Y$ is an edge.
Solution. Substituting, $Y = (a + bc)X + c\,\varepsilon_M + \varepsilon_Y$, so $\mathrm{Cov}(X, Y) = (a + bc)\mathrm{Var}(X) = (0.6 + 1.2 \cdot (-0.5)) \cdot 1 = 0.6 - 0.6 = 0$ (the two directed paths $X \to Y$ and $X \to M \to Y$ of Corollary 6.1 below cancel). X and Y are uncorrelated, hence independent, being jointly Gaussian. *Simulation* (`ex_unfaithful()`, $n = 100{,}000$ draws, seed 4): the sample correlation is $r = -0.0035$; Fisher's test statistic $z = \sqrt{n - 3}\,\mathrm{artanh}(r) = -1.112$ gives the two-sided p-value $2(1 - \Phi(1.112)) = 0.266$, so independence is not rejected. Any perturbation $a \to a + \varepsilon$ restores $\mathrm{Cov}(X, Y) = \varepsilon$ (e.g. 0.05 or 0.2). Direct and mediated effects cancel exactly — the null-set coincidence faithfulness rules out, and a trap for structure learning (Session 11).

PYTHON LAB · s06_examples.py · ex_unfaithful()

```
a + b*c = 0.00; Cov(X,Y) = 0.0000 although X -> Y is an edge
sample corr(X,Y) = -0.0035, Fisher z = -1.112, p-value = 0.266
a -> a+0.05:  Cov(X,Y) = 0.0500; a -> a+0.2:  Cov(X,Y) = 0.2000
```

PART III · MAPS AND EQUIVALENCE

6.8 Markov Equivalence

Different DAGs can assert exactly the same independencies. $X \to Y \to Z$, $X \leftarrow Y \leftarrow Z$ and $X \leftarrow Y \to Z$ all assert $X \perp\!\!\!\perp Z \mid Y$ and nothing else, whereas $X \to Y \leftarrow Z$ asserts $X \perp\!\!\!\perp Z$ (Fig. 6.9). Counting all 25 DAGs on three labelled nodes: the empty graph (1 DAG) is a class of size 1; each of the 3 one-edge skeletons gives a class of 2 DAGs (both orientations); each of the 3 two-edge skeletons gives a class of size 3 (chain, reverse chain, fork) and a v-structure class of size 1; the 6 acyclic orientations of the triangle form one class. That is $1 + 6 + 12 + 6 = 25$ DAGs in only $1 + 3 + 6 + 1 = 11$ classes.

Definition 6.7 (Markov equivalence, v-structure)**.** G_1 and G_2 are *Markov equivalent* if $\mathcal{I}(G_1) = \mathcal{I}(G_2)$. The *skeleton* of G is its undirected version. A *v-structure* (immorality) is a triple $X \to Z \leftarrow Y$ with X and Y non-adjacent.

Theorem 6.4 (Verma–Pearl characterization)**.** *Two DAGs are Markov equivalent if and only if they have the same skeleton and the same v-structures.*

Proof of necessity. Skeleton. If X, Y are adjacent, the one-edge path between them has no intermediate node and can never be blocked, so no statement $X \perp\!\!\!\perp Y \mid Z$ is in $\mathcal{I}(G)$. If they are non-adjacent, acyclicity implies that one of them, say X, is a non-descendant of Y, and $X \notin \mathrm{pa}(Y)$. We claim $X \perp\!\!\!\perp_d Y \mid \mathrm{pa}(Y)$. A path from Y that leaves through a parent P ($P \to Y$) has P as a chain or fork node (the edge $P \to Y$ points away from P), and $P \in \mathrm{pa}(Y)$: blocked. A path that leaves through a child must, in order to reach the non-descendant X, at some point stop following arrows downwards; the node where it first turns is a collider and a descendant of Y, and neither it nor any of its descendants is in $\mathrm{pa}(Y)$ (a descendant of Y that is also a parent of Y would close a directed cycle): blocked. Hence adjacency is determined by $\mathcal{I}(G)$: equivalent graphs share the skeleton.

v-structures. Let $X - Z - Y$ with X, Y non-adjacent and let S be any set with $X \perp\!\!\!\perp_d Y \mid S$ (one exists by the first part). If Z is a collider, the path $X \to Z \leftarrow Y$ is blocked only when neither Z nor its descendants lie in S, so $Z \notin S$. If Z is a chain or fork node, the path is blocked only when $Z \in S$. Thus "$Z \in S$ for *every* set S that d-separates X and Y" (non-v-structure) versus "$Z \notin S$ for every such S" (v-structure) is a property of $\mathcal{I}(G)$, and equivalent graphs share their v-structures. Sufficiency is due to Verma and Pearl; see Koller & Friedman (2009, Ch. 3) for the proof. □

An equivalence class is represented by a *CPDAG*: edges oriented alike in every member are directed, the rest undirected. Observational data identify only the CPDAG (Session 11); orienting the rest needs assumptions or interventions (Sessions 12–14).

Worked Example 6.7 (Analytical: which DAGs are equivalent?)**.** (a) Is the student network equivalent to the version with $S \to I$ instead of $I \to S$? (b) to the version with $G \to D$ instead of $D \to G$? (c) For the diamond $A \to B$, $A \to C$, $B \to D$, $C \to D$: is $B \to A$ allowed? is $D \to B$?

Solution. (a) Yes: the skeleton is unchanged and the only v-structure $D \to G \leftarrow I$ survives; S–I–G was a fork at I and becomes a chain $S \to I \to G$. (b) No: reversing $D \to G$ turns the v-structure $D \to G \leftarrow I$ into the chain $D \leftarrow G \leftarrow I$, so the graph no longer asserts $D \perp\!\!\!\perp I$

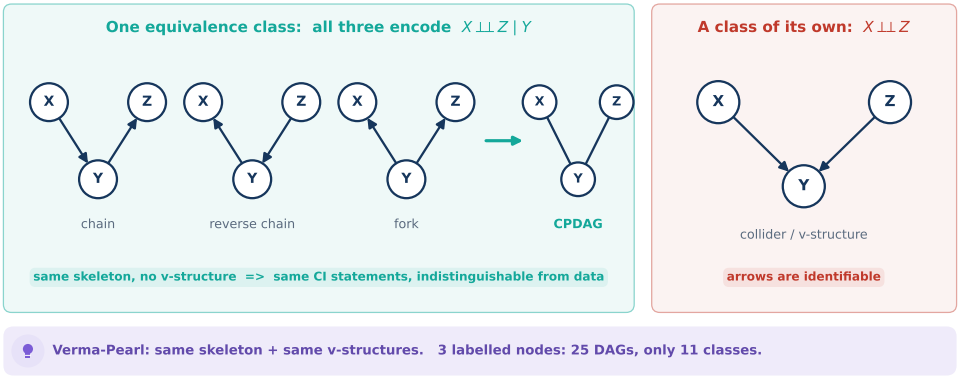


Figure 6.9. Markov equivalence. The chain, reverse chain and fork form one class, represented by an undirected CPDAG; the v-structure is alone in its class and its arrows can be recovered from observational data.

(it asserts $D \perp\!\!\!\perp I \mid G$ instead). (c) $B \to A$: skeleton and the v-structure $B \to D \leftarrow C$ are unchanged, so yes. $D \to B$: the v-structure at D disappears and a new one $A \to B \leftarrow D$ appears; not equivalent.

PYTHON LAB · s06_examples.py · ex_markov_equivalence()

```
chain~rev.chain True, chain~fork True, chain~collider False; student~(S->I) True, ~(G->D)
False
diamond~(B->A) True, ~(D->B) False; 3 labelled nodes:  25 DAGs, 11 classes, sizes
[1,1,1,1,2,2,2,3,3,3,6]
```

PART IV

Gaussian Networks, Likelihood and Testing

Covariances from paths, CPTs from counts, edges from tests

PART IV · MATHEMATICAL STATISTICS

6.9 Linear-Gaussian Bayesian Networks

Collect the linear-Gaussian CPDs (6.4) of all nodes into one vector equation. Order the nodes topologically and let B be the $n \times n$ matrix with $B_{ij} = b_{ij}$ if $X_j \to X_i$ and 0 otherwise; B is strictly lower triangular. Then

$$\mathbf{X} = \boldsymbol{\beta}_0 + B\mathbf{X} + \boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, D), \quad D = \operatorname{diag}(d_1, \dots, d_n). \tag{6.6}$$

Theorem 6.5 (A linear-Gaussian BN is a multivariate normal)**.** *Under* (6.6), $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$ *with*

$$\boldsymbol{\mu} = (I - B)^{-1}\boldsymbol{\beta}_0, \qquad \Sigma = (I - B)^{-1} D\, (I - B)^{-\top}, \qquad \Omega = \Sigma^{-1} = (I - B)^{\top} D^{-1} (I - B).$$

Proof. $I - B$ is unit lower triangular, so $\det(I - B) = 1$ and it is invertible; solving (6.6) gives

$\mathbf{X} = (I-B)^{-1}(\boldsymbol{\beta}_0+\boldsymbol{\varepsilon})$, an affine map of a Gaussian vector, hence Gaussian. Its mean is $(I-B)^{-1}\boldsymbol{\beta}_0$ since $\mathbb{E}\boldsymbol{\varepsilon} = \mathbf{0}$, and its covariance is $(I-B)^{-1}\mathrm{Cov}(\boldsymbol{\varepsilon})(I-B)^{-\top}$. Inverting the product gives Ω. □

Corollary 6.1 (Path tracing / trek rule)**.** *Let $\tau_{ik} = \left[(I-B)^{-1}\right]_{ik}$. Then $\tau_{ii} = 1$ and, for $i \neq k$, τ_{ik} is the sum over all directed paths $X_k \to \cdots \to X_i$ of the product of the edge weights. Consequently*

$$\mathrm{Cov}(X_i, X_j) = \sum_{k=1}^{n} d_k\, \tau_{ik}\, \tau_{jk},$$

a sum over common ancestors X_k (including X_i or X_j themselves) of "treks" $X_i \leftarrow \cdots \leftarrow X_k \to \cdots \to X_j$. In particular, if X_i and X_j are d-separated given $\emptyset$ (no common ancestor), they are uncorrelated.

Proof. B is strictly lower triangular, hence nilpotent ($B^n = 0$), so $(I-B)^{-1} = \sum_{m=0}^{n-1} B^m$. By induction on m, $(B^m)_{ik} = \sum b_{ij_{m-1}} \cdots b_{j_1 k}$ over all directed paths of length m from X_k to X_i. Summing over m gives τ_{ik}. The covariance formula is the (i,j) entry of $(I-B)^{-1}D(I-B)^{-\top}$. If X_i, X_j have no common ancestor then $\tau_{ik}\tau_{jk} = 0$ for every k. □

MATHEMATICAL STATISTICS · Zeros of the precision matrix and the moral graph

From $\Omega = (I-B)^\top D^{-1}(I-B)$ we get $\Omega_{ij} = \sum_k (I-B)_{ki}(I-B)_{kj}/d_k$. For $i \neq j$ a term is non-zero only if $k \in \{i, j\}$ and the other node is a parent of k, or if X_i and X_j are both parents of the same X_k. Hence $\Omega_{ij} \neq 0$ only when X_i, X_j are adjacent or co-parents — adjacent in the *moral graph*. With Session 5's result $\Omega_{ij} = 0 \iff X_i \perp\!\!\!\perp X_j \mid \text{rest}$ (there the precision matrix was written Θ; here Θ denotes the CPD parameters, so we write Ω), this says: a node is independent of everything outside its Markov blanket given the rest, the Gaussian face of Theorem 6.3. Moralization returns in Sessions 7–8.

Worked Example 6.8 (A four-node linear-Gaussian network)**.** Illustrative model: $X_1 \sim \mathcal{N}(1,1)$, $X_2 = 0.8X_1 + \varepsilon_2$, $X_3 = 2 - 0.5X_1 + \varepsilon_3$, $X_4 = -1 + 1.0X_2 + 0.6X_3 + \varepsilon_4$, with $d = (1, 0.36, 0.75, 0.5)$. Find $\boldsymbol{\mu}$, Σ and the zero pattern of Ω.

Solution. Means propagate along edges: $\mu_1 = 1$, $\mu_2 = 0.8 \cdot 1 = 0.8$, $\mu_3 = 2 - 0.5 \cdot 1 = 1.5$, $\mu_4 = -1 + 1.0 \cdot 0.8 + 0.6 \cdot 1.5 = -1 + 0.8 + 0.9 = 0.7$.

Covariances by path tracing (Corollary 6.1); the path products are $\tau_{21} = 0.8$, $\tau_{31} = -0.5$, $\tau_{42} = 1.0$, $\tau_{43} = 0.6$ and $\tau_{41} = 0.8 \cdot 1.0 + (-0.5) \cdot 0.6 = 0.5$ (two directed paths from X_1 to X_4):

$$\begin{aligned}
\mathrm{Var}X_2 &= d_1\tau_{21}^2 + d_2 = 0.64 + 0.36 = 1, & \mathrm{Var}X_3 &= d_1\tau_{31}^2 + d_3 = 0.25 + 0.75 = 1,\\
\mathrm{Cov}(X_1, X_2) &= d_1\tau_{21} = 0.8, & \mathrm{Cov}(X_1, X_3) &= d_1\tau_{31} = -0.5,\\
\mathrm{Cov}(X_1, X_4) &= d_1\tau_{41} = 0.5, & \mathrm{Cov}(X_2, X_3) &= d_1\tau_{21}\tau_{31} = -0.4,
\end{aligned}$$

$$\begin{aligned}
\mathrm{Cov}(X_2, X_4) &= d_1\tau_{21}\tau_{41} + d_2\tau_{42} = 0.4 + 0.36 = 0.76,\\
\mathrm{Cov}(X_3, X_4) &= d_1\tau_{31}\tau_{41} + d_3\tau_{43} = -0.25 + 0.45 = 0.2,
\end{aligned}$$

$\mathrm{Var}X_4 = d_1\tau_{41}^2 + d_2\tau_{42}^2 + d_3\tau_{43}^2 + d_4 = 1 \cdot 0.25 + 0.36 \cdot 1 + 0.75 \cdot 0.36 + 0.5 = 0.25 + 0.36 + 0.27 + 0.5 = 1.38.$

$\mathrm{Cov}(X_2, X_3) = -0.4$ is a trek through the common ancestor X_1 although X_2, X_3 are not adjacent.

Precision. By $\Omega_{ij} = \sum_k (I-B)_{ki}(I-B)_{kj}/d_k$, with $(I-B)_{kk} = 1$ and $(I-B)_{kj} = -b_{kj}$, the off-diagonal entries are $\Omega_{12} = -b_{21}/d_2 = -0.8/0.36 \approx -2.22$, $\Omega_{13} = -b_{31}/d_3 = 0.5/0.75 \approx 0.67$,

$\Omega_{24} = -b_{42}/d_4 = -1.0/0.5 = -2$, $\Omega_{34} = -b_{43}/d_4 = -0.6/0.5 = -1.2$, $\Omega_{23} = b_{42}b_{43}/d_4 = 1.0 \cdot 0.6/0.5 = 1.2$ and $\Omega_{14} = 0$ (no k has both X_1 and X_4 in $\{X_k\} \cup \mathrm{pa}(X_k)$). So the only zero of Ω is Ω_{14}, while $\Omega_{23} \neq 0$ because X_2 and X_3 are married through their child X_4 (Fig. 6.10). *Simulation check* (seed 1): 200,000 forward samples reproduce every entry of Σ to within 0.0062.

PYTHON LAB · s06_examples.py · ex_linear_gaussian()

```
mu = [1.  0.8 1.5 0.7]  Sigma[0,3] = 0.50, Sigma[1,2] = -0.40, Sigma[3,3] = 1.38
max |S_hat - S| = 0.0062  precision[0,3] = 0.00, precision[1,2] = 1.20
precision rows:  [3.11 -2.22 0.67 0.]  [-2.22 4.78 1.2 -2.]  [0.67 1.2 2.05 -1.2] [0.  -2.
-1.2 2.]
```

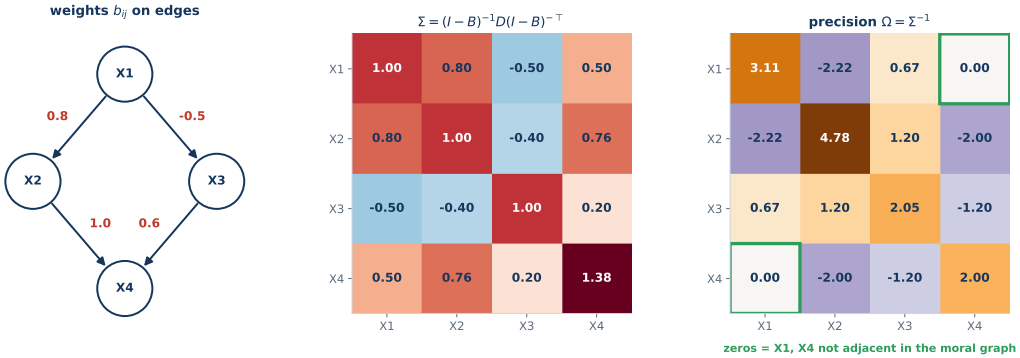


Figure 6.10. A linear-Gaussian BN (left), its implied covariance (centre) and precision matrix (right). Covariances are sums of path products; zeros of the precision mark non-adjacency in the moral graph.

Worked Example 6.9 (Analytical: explaining away in a Gaussian collider)**.** Let $X \sim \mathcal{N}(0, \sigma_x^2)$ and $Y \sim \mathcal{N}(0, \sigma_y^2)$ be independent and $Z = X + Y + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$. Derive $\mathrm{Cov}(X, Y \mid Z)$. *Solution.* By Corollary 6.1, $\mathrm{Var} Z = s := \sigma_x^2 + \sigma_y^2 + \sigma^2$, $\mathrm{Cov}(X, Z) = \sigma_x^2$, $\mathrm{Cov}(Y, Z) = \sigma_y^2$, $\mathrm{Cov}(X, Y) = 0$. The Gaussian conditioning (Schur complement) formula gives

$$\mathrm{Cov}(X, Y \mid Z) = 0 - \frac{\sigma_x^2 \sigma_y^2}{s} < 0, \qquad \mathrm{Var}(X \mid Z) = \sigma_x^2 - \frac{\sigma_x^4}{s}.$$

(For a Gaussian vector, $\mathrm{Cov}(\mathbf{a} \mid \mathbf{b}) = \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}$; here $\mathbf{a} = (X, Y)$, $\mathbf{b} = Z$.) Conditioning on the collider makes the causes *negatively* correlated, whatever the variances: if Z is high and X is low, Y must be high. With $\sigma_x^2 = \sigma_y^2 = 1$, $\sigma^2 = 0.5$: $s = 1 + 1 + 0.5 = 2.5$, $\mathrm{Cov}(X, Y \mid Z) = -1 \cdot 1/2.5 = -0.4$, $\mathrm{Var}(X \mid Z) = \mathrm{Var}(Y \mid Z) = 1 - 1/2.5 = 0.6$, so the partial correlation is $-0.4/0.6 = -0.6667$. *Simulation* (`ex_gaussian_explaining_away()`, 200,000 draws, seed 3): the sample correlation of X, Y is 0.0025, the sample partial correlation given Z (correlation of the residuals of X and Y regressed on Z) is -0.6649, and among the 17.2% of units with $Z > 1.5$ (selection, a softer form of conditioning) the correlation is -0.4605. This is Berkson's paradox, the subject of Session 12.

PYTHON LAB · s06_examples.py · ex_gaussian_explaining_away()

```
Sigma_{XY|Z} = [[0.6 -0.4] [-0.4 0.6]]; formula Cov(X,Y|Z) = -0.4000; partial corr = -0.6667
simulated:  corr(X,Y) = 0.0025, partial corr given Z = -0.6649
selection Z > 1.5 (0.172 of the sample):  corr(X,Y | sel) = -0.4605
```

PART IV · MATHEMATICAL STATISTICS

6.10 Likelihood, Parameters and Tests

Theorem 6.6 (The likelihood factorizes over CPTs; MLE = normalized counts)**.** *Let* $\mathcal{D} = \{\mathbf{x}[1], \dots, \mathbf{x}[M]\}$ *be i.i.d. complete observations from a discrete BN with tabular CPDs* $\theta^{(i)}_{x|u} = P(X_i = x \mid \mathrm{pa}(X_i) = u)$*. Let* $N_i(x,u)$ *be the number of records with* $X_i = x$*,* $\mathrm{pa}(X_i) = u$*, and* $N_i(u) = \sum_x N_i(x,u)$*. Then*

$$\ell(\theta) = \sum_{i=1}^{n} \sum_{u} \sum_{x} N_i(x,u) \log \theta^{(i)}_{x|u}, \qquad \hat{\theta}^{(i)}_{x|u} = \frac{N_i(x,u)}{N_i(u)} \quad \text{whenever } N_i(u) > 0.$$

Proof. By independence and (6.1), $\ell(\theta) = \sum_m \sum_i \log \theta^{(i)}_{x_i[m]|u_i[m]}$; grouping identical (x,u) pairs gives the stated form. Each block (i,u) involves only the row $\theta^{(i)}_{\cdot|u}$, and rows are constrained separately (each lies on a simplex), so ℓ is maximized row by row. For one row, with $\hat{p}(x) = N_i(x,u)/N_i(u)$,

$$\sum_x N_i(x,u) \log \theta_x = N_i(u) \Big[\sum_x \hat{p}(x) \log \hat{p}(x) - \mathrm{KL}(\hat{p} \,\|\, \theta) \Big],$$

and Gibbs' inequality $\mathrm{KL} \geq 0$, with equality iff $\theta = \hat{p}$, gives the maximizer. □

Worked Example 6.10 (Learning the grade CPT from 5000 students)**.** *Simulation* (`ex_mle_counts()`, seed 2024): generate $M = 5000$ students from Fig. 6.2 by *forward (ancestral) sampling* — draw D and I from their priors, then G from the row $P(G \mid i, d)$, S from $P(S \mid i)$ and L from $P(L \mid g)$ — and estimate $P(G \mid I, D)$ by counting. The parent configuration (i^1, d^1) occurs $N(u) = 633$ times (expected $5000 \cdot 0.3 \cdot 0.4 = 600$) with grade counts $(325, 185, 123)$, so $\hat{\theta} = (325, 185, 123)/633 = (0.513, 0.292, 0.194)$ against the true $(0.5, 0.3, 0.2)$. The common configuration (i^0, d^0) occurs 2079 times with counts $(630, 815, 634)$, giving $(0.303, 0.392, 0.305)$ against $(0.3, 0.4, 0.3)$. Evaluated at the true CPTs, the log-likelihood of the sample splits into the five local terms -3365.869 (D), -3098.381 (I), -4259.201 (G), -1436.493 (S) and -1647.602 (L), whose sum -13807.546 equals the total log-likelihood, as the theorem says.

PYTHON LAB · s06_examples.py · ex_mle_counts()

```
('i0','d0'):  2079 [0.303 0.392 0.305] true [0.3, 0.4, 0.3] counts [630, 815, 634]
('i1','d1'):  633 [0.513 0.292 0.194] true [0.5, 0.3, 0.2] counts [325, 185, 123]
local log-likelihoods at the true CPTs:  D -3365.869, I -3098.381, G -4259.201, S -1436.493,
L -1647.602
sum of local terms = -13807.546; total log-likelihood = -13807.546; s.e.  = 0.020
```

MATHEMATICAL STATISTICS · Sampling variability of a CPT entry

Conditional on $N_i(u)$, the count $N_i(x,u)$ is $\mathrm{Bin}(N_i(u), \theta_{x|u})$, so $\hat{\theta}_{x|u}$ is unbiased with variance $\theta(1-\theta)/N_i(u)$. Since $N_i(u)/M \xrightarrow{p} P(u)$, the delta method gives $\sqrt{M}(\hat{\theta}_{x|u} - \theta_{x|u}) \xrightarrow{d} \mathcal{N}\big(0, \theta(1-\theta)/P(u)\big)$. Rare parent configurations give noisy rows; each extra parent multiplies the rows and divides the data per row. In the example, $\hat{\theta}(g^1 \mid i^1, d^1) = 0.513$ has standard error $\sqrt{0.513 \cdot 0.487/633} = 0.020$. This is the statistical reason for sparse graphs and compact CPDs, and the starting point of Bayesian smoothing in Session 10.

MATHEMATICAL STATISTICS · Model dimension and testing an edge

Formula (6.2) is the model dimension. Removing edges imposes CI constraints, tested by the likelihood-ratio statistic $G^2 = 2\sum_{\text{cells}} O \log(O/E)$, where O are the observed counts and $E = (\text{row total})(\text{column total})/(\text{table total})$ the counts expected under independence; it is asymptotically χ^2 with df equal to the drop in dimension (Wilks). In the $M = 5000$ sampled students of Worked Example 6.10, the $D \times S$ table (rows d^0, d^1; columns s^0, s^1) is $O = \left[\begin{smallmatrix} 2157 & 841 \\ 1400 & 602 \end{smallmatrix}\right]$ with $E = \left[\begin{smallmatrix} 2132.78 & 865.22 \\ 1424.22 & 577.78 \end{smallmatrix}\right]$ (e.g. $2998 \cdot 3557/5000 = 2132.78$), giving $G^2 = 2.375$ on $(2-1)(2-1) = 1$ df, $p = 0.123$ — not rejected, as d-separation predicts. For $D \perp\!\!\!\perp S \mid G$ the same statistic is computed within each grade and summed: $G^2 = 72.15{+}84.63{+}25.61 = 182.39$ on $1 \cdot 1 \cdot 3 = 3$ df, $p = 2.69 \times 10^{-39}$ — rejected, the collider at work. Sequences of such tests are the engine of the PC algorithm (Session 11).

PYTHON LAB · s06_examples.py · ex_ci_tests()

```
observed D x S counts (rows d0,d1; cols s0,s1):  [[2157, 841], [1400, 602]]
D vs S: G = 2.375, df = 1, p = 0.123
stratum g1:  G = 72.15; stratum g2:  G = 84.63; stratum g3:  G = 25.61
D vs S | G: G = 182.39, df = 3, p = 2.69e-39
```

Remark 6.1 (Noisy-OR likelihood). With $w_i = -\log(1-\lambda_i)$ and $t = w_0 + \sum_i w_i x_i$, (6.3) reads $P(Y = 0 \mid \mathbf{x}) = e^{-t}$, so one record contributes $(1-y)(-t) + y\log(1-e^{-t})$. The first term is linear and $f(t) = \log(1-e^{-t})$ has $f''(t) = -e^t/(e^t-1)^2 < 0$, so the log-likelihood (a sum of such terms, with t linear in $\mathbf{w}$) is *concave* in $\mathbf{w}$ on the region $\mathbf{w} \geq 0$, and any local maximizer is a global MLE. *Simulation* (`ex_noisy_or_mle()`, seed 6): 4000 records with independent causes $X_1, X_2, X_3 \sim \text{Bern}(0.4), \text{Bern}(0.3), \text{Bern}(0.5)$ and Y drawn from the noisy-OR with true $(\lambda_0, \ldots, \lambda_3) = (0.05, 0.5, 0.7, 0.3)$; L-BFGS-B on $\mathbf{w}$ started from $\mathbf{w} = (0.5, 0.5, 0.5, 0.5)$ and from $\mathbf{w} = (2, 0.01, 3, 0.1)$ returns the same estimate $\hat{\lambda} = 1 - e^{-\hat{\mathbf{w}}} = (0.052, 0.517, 0.691, 0.297)$.

PYTHON LAB · s06_examples.py · ex_noisy_or_mle()

```
true (leak, l1, l2, l3) = [0.05 0.5 0.7 0.3]
MLE (leak, l1, l2, l3) = [0.052 0.517 0.691 0.297]; from another start:  the same
```

PART IV · MATHEMATICAL STATISTICS

6.11 Worked Examples

Worked Example 6.11 (Is $D \perp\!\!\!\perp S$?)**.** In the student network (Fig. 6.2), is Difficulty d-separated from SAT with no evidence, $Z = \emptyset$?

Solution. The only path between D and S is $D \to G \leftarrow I \to S$. The intermediate node G is a collider on this path (two arrowheads meet at G), and $G \notin Z = \emptyset$, and no descendant of G (namely L) is in Z either. An unobserved collider blocks a path, so this single path is blocked, and since it is the only path, $D \perp\!\!\!\perp_d S \mid \emptyset$. Difficulty and SAT score are independent — consistent with the intuitive causal story that they share no cause and no evidence has been observed to make them "compete" as explanations of anything. Numerically, summing out I (and G, L, whose CPT rows sum to one), $P(d^1, s^1) = P(d^1)\sum_i P(i)P(s^1 \mid i) = 0.4\,(0.7 \cdot 0.05 + 0.3 \cdot 0.8) = 0.4 \cdot 0.275 = 0.110$, which equals $P(d^1)P(s^1)$ because $P(s^1) = 0.275$.

Worked Example 6.12 (Conditioning on a collider creates dependence)**.** Now suppose we *do* observe the grade, $Z = \{G\}$. Is $D \perp\!\!\!\perp_d S \mid G$? See Fig. 6.6.
Solution. The same path $D \to G \leftarrow I \to S$ now has its collider G observed ($G \in Z$). An observed collider *opens* the path rather than blocking it. Since this is the only path and it is now active, $D \not\perp\!\!\!\perp S \mid G$. Intuitively: once you know a student got a C, the poor grade needs an explanation — a hard course or low intelligence. A high SAT score is evidence of high intelligence, which removes the second explanation and makes the first more likely; so S becomes informative about D. With the CPTs, summing over I (the L factor sums to one):

$$\begin{aligned} P(d^1, g^3) &= 0.4\,(0.7 \cdot 0.70 + 0.3 \cdot 0.20) = 0.4 \cdot 0.55 = 0.22, \\ P(d^0, g^3) &= 0.6\,(0.7 \cdot 0.30 + 0.3 \cdot 0.02) = 0.6 \cdot 0.216 = 0.1296, \\ P(d^1, g^3, s^1) &= 0.4\,(0.7 \cdot 0.70 \cdot 0.05 + 0.3 \cdot 0.20 \cdot 0.8) = 0.4 \cdot 0.0725 = 0.029, \\ P(d^0, g^3, s^1) &= 0.6\,(0.7 \cdot 0.30 \cdot 0.05 + 0.3 \cdot 0.02 \cdot 0.8) = 0.6 \cdot 0.0153 = 0.00918, \end{aligned}$$

so $P(d^1 \mid g^3) = 0.22/(0.22 + 0.1296) = 0.629$ but $P(d^1 \mid g^3, s^1) = 0.029/(0.029 + 0.00918) = 0.760$. This "explaining away" phenomenon is a direct generalization of Session 5's XOR example (conditioning on the common effect $Z = X \oplus Y$ made two independent coins dependent), and it is also the graphical seed of an important warning: naively "controlling for everything you can measure" can *create* spurious associations rather than remove them — a theme we revisit sharply in Session 12.

PYTHON LAB · `s06_examples.py` · `ex_dsep_checks()`

```
D dsep S | {}:  True; | {G}:  False; | {L}:  False; P(s1)=0.275 P(d1,s1)=0.110
P(d1)P(s1)=0.110
P(d1, g3) = 0.2200 P(d1, g3, s1) = 0.02900; P(d0, g3) = 0.1296 P(d0, g3, s1) = 0.00918
P(d1|g3)=0.629 but P(d1|g3,s1)=0.760
```

Worked Example 6.13 (Local Markov property in the medical network)**.** Recall Session 5's disease-symptom model, now drawn as a DAG with edges $D \to S_1$, $D \to S_2$, $D \to S_3$ (a fork at D with three children). Use d-separation to confirm $S_1 \perp\!\!\!\perp S_2 \mid D$.
Solution. The only path between S_1 and S_2 is $S_1 \leftarrow D \to S_2$, a fork with middle node D. Since $D \in Z = \{D\}$, the fork is blocked, so $S_1 \perp\!\!\!\perp_d S_2 \mid D$, confirming by the graphical criterion exactly the naive-Bayes assumption we asserted by hand (via algebra) in Session 5. This is the payoff promised at the top of this session: what took an explicit CI definition and direct verification last time is now a two-second visual check.

Additional Worked Examples (Instructor Supplement)

Worked Example 6.14 (Conditioning on a *descendant* of a collider also opens the path)**.** The d-separation definition above says a collider blocks a path unless *the collider or one of its descendants* is in Z — the worked examples above only ever conditioned on the collider itself. Consider Rain R, Sprinkler S (independent causes), WetGrass G (a collider: $R \to G \leftarrow S$), and SlipperyFloor F, a further effect $G \to F$. Is $R \perp\!\!\!\perp_d S \mid F$?
Solution. The only path is $R \to G \leftarrow S$, with collider G. Here $Z = \{F\}$: $G \notin Z$, but F *is* a descendant of G, and $F \in Z$. By that definition, a collider blocks a path only if *neither* it nor any of its descendants is in Z — that condition fails here, so the path is **active**, and

$R \not\perp\!\!\!\perp S \mid F$. Intuitively: learning the floor is slippery makes rain and sprinkler use *both* more plausible explanations, but if you additionally learn it was *not* raining, the sprinkler becomes more likely to have been on — exactly the explaining-away logic of Worked Example 6.12, now triggered by evidence on a downstream effect rather than on the collider itself. This is the clause students most often forget when applying d-separation, and it is exactly why post-treatment variables are dangerous to condition on — a warning Session 14's backdoor criterion will make precise.

Worked Example 6.15 (A second network for practice: alarm, burglary, earthquake)**.** Consider the classic network: Burglary B and Earthquake E (independent, unconditionally) both cause Alarm A ($B \to A \leftarrow E$, a collider), and A causes two calls, JohnCalls J and MaryCalls M ($A \to J$, $A \to M$). Answer, with justification: (a) $B \perp\!\!\!\perp_d E \mid \emptyset$? (b) $B \perp\!\!\!\perp_d E \mid A$? (c) $J \perp\!\!\!\perp_d M \mid \emptyset$? (d) $J \perp\!\!\!\perp_d M \mid A$?

Solution. (a) The only path $B \to A \leftarrow E$ has collider $A \notin \emptyset$ and no descendant of A is in $\emptyset$ either, so the path is blocked: $B \perp\!\!\!\perp_d E \mid \emptyset$ — burglary and earthquakes are unconditionally unrelated, as expected. (b) Now $A \in Z = \{A\}$, opening the collider: $B \not\perp\!\!\!\perp E \mid A$ (if the alarm went off and it wasn't a burglary, an earthquake becomes more plausible — textbook explaining-away). (c) The only path between J and M is $J \leftarrow A \to M$, a *fork* with fork-node $A \notin \emptyset$, so it is unblocked: $J \not\perp\!\!\!\perp M \mid \emptyset$ (both calls are correlated simply because both depend on whether the alarm rang). (d) With $A \in Z$, the fork *is* now blocked: $J \perp\!\!\!\perp_d M \mid A$ — once you already know whether the alarm sounded, John's and Mary's calling behaviors carry no further information about each other. Notice (b) and (d) are opposite effects of the same act of conditioning on A: it opens the collider path for (B, E) while closing the fork path for (J, M) — the same node can play both roles simultaneously for different pairs of variables in a larger graph. The Bayes-ball runs of `ex_bayes_ball()` confirm (a) and (b): from B with no evidence the ball reaches A, J, M but not E, and with $Z = \{A\}$ it reaches E. The run drawn in Fig. 6.7 uses $Z = \{J\}$: J is a descendant of the collider A, so E is reached and $B \not\perp\!\!\!\perp E \mid J$, the clause of Worked Example 6.14.

PYTHON LAB · `s06_examples.py` · `ex_dsep_checks()`

```
medical:  S1 dsep S2 | {Dis}:  True; rain:  R dsep S | {}:  True, R dsep S | {F}:  False
alarm:  B dsep E | {}:  True; B dsep E | {A}:  False; J dsep M | {}:  False; J dsep M | {A}:
True
```

Worked Example 6.16 (Deriving a chain's independence directly from the factorization)**.** For the simple chain $A \to B \to C$, derive $A \perp\!\!\!\perp C \mid B$ directly from the BN chain rule (Definition 6.1), without invoking d-separation as a black box — reproducing, on a minimal example, on a minimal example, the "factorize, then split" step (Lemma B) that the appendix proof of Theorem 6.2 performs in general.

Solution. The chain rule gives $P(a, b, c) = P(a)P(b \mid a)P(c \mid b)$ (since $\mathrm{pa}(A) = \emptyset$, $\mathrm{pa}(B) = \{A\}$, $\mathrm{pa}(C) = \{B\}$). Then

$$P(a, c \mid b) = \frac{P(a, b, c)}{P(b)} = \frac{P(a)P(b \mid a)P(c \mid b)}{P(b)} = \left(\frac{P(a)P(b \mid a)}{P(b)}\right) P(c \mid b) = P(a \mid b)\, P(c \mid b),$$

using Bayes' rule to identify the parenthesized term as $P(a \mid b)$. This is exactly Session 5's definition of conditional independence, confirming $A \perp\!\!\!\perp C \mid B$ purely algebraically — and it is precisely Session 5's factorized-form proposition ($g(a, b)h(c, b)$ with $g(a, b) = P(a)P(b \mid a)$, $h(c, b) = P(c \mid b)$) applied to this specific factorization. D-separation's chain rule ("a chain

node, once observed, blocks the path") is nothing more than this five-line computation, generalized and automated so it never needs to be redone by hand for every new graph. Numerical check with illustrative CPTs $P(A{=}1) = 0.7$, $P(B{=}1 \mid A{=}0) = 0.1$, $P(B{=}1 \mid A{=}1) = 0.8$, $P(C{=}1 \mid B{=}0) = 0.4$, $P(C{=}1 \mid B{=}1) = 0.9$: $P(B{=}1) = 0.3 \cdot 0.1 + 0.7 \cdot 0.8 = 0.59$, $P(A{=}1 \mid B{=}1) = 0.56/0.59 = 0.9492$, and $P(A{=}1, C{=}1 \mid B{=}1) = 0.7 \cdot 0.8 \cdot 0.9/0.59 = 0.8542 = 0.9492 \cdot 0.9$, as the identity requires (the script checks all cells, for both values of b, to machine precision). Marginally, however, $P(C{=}1) = 0.41 \cdot 0.4 + 0.59 \cdot 0.9 = 0.695$ and $P(A{=}1, C{=}1) = 0.7\,(0.2 \cdot 0.4 + 0.8 \cdot 0.9) = 0.56 \neq P(A{=}1)P(C{=}1) = 0.4865$: A and C remain dependent (largest cell difference 0.0735).

PYTHON LAB · s06_examples.py · ex_chain_numeric()

```
max |P(a,c|b)-P(a|b)P(c|b)|:  b=0 5.55e-17, b=1 1.11e-16
P(b=1) = 0.59; P(a=1|b=1) = 0.9492; P(c=1|b=1) = 0.90; P(a=1,c=1|b=1) = 0.8542
P(c=1) = 0.695; P(a=1,c=1) = 0.5600; P(a=1)P(c=1) = 0.4865; marginal max |P(a,c)-P(a)P(c)| =
0.0735
```

PART V

Bayesian Networks in Agents, RL and Multi-Agent Systems

The same DAG is a world model, a transition model and a distributed belief

PART V · AI CONNECTIONS

6.12 Bayesian Networks as World Models for Agents

A rational, model-based agent (course sessions 1 and 25) maps percepts to actions through an internal model of the unobserved world. A BN is the simplest explicit, inspectable such model: its DAG says what affects what, its CPTs say how strongly, d-separation says which observations are relevant to which hidden variables, and updating the agent's state on a new percept is a posterior computation.

6.12.1 Agentic AI: which tool should the agent call next?

AI CONNECTION · Agentic AI

Tool-using LLM agents (course session 26) must decide *which* tool to call. When the uncertainty is over a few discrete causes, a small BN behind the agent turns this into a computation: score each tool by the expected reduction in uncertainty about the causes, call the best one, enter its result as evidence, repeat (Fig. 6.11). The language model handles dialogue and tool syntax; the BN handles the probabilistic bookkeeping.

Worked Example 6.17 (Deployment-failure diagnosis (hypothetical numbers))**.** Causes: configuration error C (prior 0.30) and network outage N (prior 0.10), independent; symptom F

= deployment fails, noisy-OR with $\lambda_0 = 0.02$, $\lambda_C = 0.8$, $\lambda_N = 0.9$. Tools: `read_logs` reports a config problem with sensitivity 0.90 and false-positive rate 0.10; `ping` reports an outage with sensitivity 0.95, false-positive rate 0.05. The user reports $F = 1$. Which tool first?
Each tool's report depends only on its own cause: $P(\text{logs}{=}1 \mid C{=}1) = 0.90$, $P(\text{logs}{=}1 \mid C{=}0) = 0.10$, $P(\text{ping}{=}1 \mid N{=}1) = 0.95$, $P(\text{ping}{=}1 \mid N{=}0) = 0.05$. Score a tool by its expected information gain $\text{IG} = H(C, N \mid F{=}1) - \sum_o P(o \mid F{=}1)\, H(C, N \mid F{=}1, o)$, with entropies in bits.

Solution. (1) Noisy-OR: $P(F{=}1 \mid c, n) = 1 - 0.98 \cdot 0.2^c \cdot 0.1^n$, i.e. 0.02, 0.902, 0.804, 0.9804 for $(c, n) = (0,0), (0,1), (1,0), (1,1)$. Multiplying by the priors $P(c)P(n)$ gives the joint weights $0.7 \cdot 0.9 \cdot 0.02 = 0.0126$, $0.7 \cdot 0.1 \cdot 0.902 = 0.06314$, $0.3 \cdot 0.9 \cdot 0.804 = 0.21708$, $0.3 \cdot 0.1 \cdot 0.9804 = 0.029412$, whose sum is $P(F{=}1) = 0.3222$. (2) Dividing, the posterior over (c, n) is (0.0391, 0.1959, 0.6737, 0.0913), so $P(C{=}1 \mid F) = 0.6737 + 0.0913 = 0.7650$, $P(N{=}1 \mid F) = 0.1959 + 0.0913 = 0.2872$, and $H(C, N \mid F{=}1) = -\sum p \log_2 p = 1.34277$ bits. (3) `read_logs`: $P(\text{logs}{=}1 \mid F) = 0.9 \cdot 0.7650 + 0.1 \cdot 0.2350 = 0.7120$; the posteriors after logs=1 and logs=0 (multiply each state by its likelihood 0.9/0.1 resp. 0.1/0.9 and renormalize) have entropies 0.7407 and 1.4520, so the expected entropy is $0.7120 \cdot 0.7407 + 0.2880 \cdot 1.4520 = 0.94559$ and $\text{IG} = 1.34277 - 0.94559 = 0.39718$ bits. (4) `ping`: $P(\text{ping}{=}1 \mid F) = 0.95 \cdot 0.2872 + 0.05 \cdot 0.7128 = 0.3085$; posterior entropies 1.3495 and 0.4648, expected entropy $0.3085 \cdot 1.3495 + 0.6915 \cdot 0.4648 = 0.73773$, $\text{IG} = 1.34277 - 0.73773 = 0.60504$ bits.
`ping` wins: although the config error is the likelier cause, `ping` is sharper and targets the more uncertain hypothesis. If `ping` reports an outage, the posterior becomes $(0.0063, 0.6034, 0.1092, 0.2811)$ and the config error is explained away: $P(C{=}1 \mid F, \text{ping}{=}1) = 0.1092 + 0.2811 = 0.3903$. Information gain is a utility-free proxy for the *value of information* (course session 21).

PYTHON LAB · `s06_examples.py` · `ex_agent_voi()`

```
joint P(c,n,F=1) = [0.0126, 0.06314, 0.21708, 0.029412]; P(F=1) = 0.3222
P(C=1|F=1) = 0.7650, P(N=1|F=1) = 0.2872; H(C,N|F=1) = 1.34277 bits
read_logs:  entropies 0.7407/1.4520, expected 0.94559, information gain = 0.39718 bits
ping:  entropies 1.3495/0.4648, expected 0.73773, information gain = 0.60504 bits
P(C=1 | F=1, ping=1) = 0.3903
```

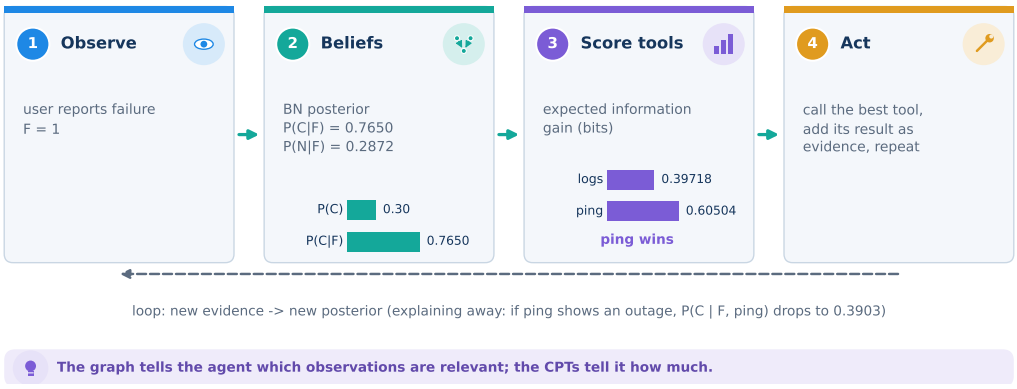


Figure 6.11. An agent loop with a BN as diagnostic world model: observe, update the posterior, score candidate tools by expected information gain, act, and feed the result back as evidence.

LINK TO COURSE SESSION 26 · Agentic AI

A BN makes an agent's assumptions auditable: one reads off which tool outputs are conditionally independent given the causes (their evidence multiplies) and which are not (shared upstream failures). Reflection can be implemented as "query the BN, compare to the plan, re-plan".

6.12.2 Reinforcement learning: dynamic Bayesian networks

AI CONNECTION · Reinforcement Learning

An MDP's transition model $P(s_{t+1} \mid s_t, a_t)$ is a CPT. When the state is a vector of variables, a *dynamic Bayesian network* (DBN) represents it compactly: a two-slice template (2-TBN) gives each next-step variable a few parents in the current slice, $P(\mathbf{x}^{t+1} \mid \mathbf{x}^t, a^t) = \prod_i P(x_i^{t+1} \mid \mathrm{pa}(X_i^{t+1}))$, and unrolling the template over time gives the full BN (Fig. 6.12). HMMs and POMDPs are the special cases with hidden states and observation CPDs $P(o_t \mid s_t)$. The Markov property itself is a d-separation statement: in the unrolled graph $S_{t+1} \perp\!\!\!\perp_d \{S_{0:t-1}, A_{0:t-1}\} \mid S_t, A_t$.

Worked Example 6.18 (Factored versus flat transition models)**.** A robot's state consists of $n = 10$ binary variables, each with at most 2 parents in the previous slice. A flat transition table needs $2^{10}(2^{10} - 1) = 1{,}047{,}552$ parameters per action; the DBN needs $10 \cdot 2^2 = 40$. With $n = 20$: about 1.1×10^{12} versus 80. Factored MDPs (course session 20) exploit this structure.

PYTHON LAB · s06_examples.py · ex_factored_mdp()

```
n=10:  flat 1,047,552 vs DBN 40 params per action; n=20:  flat 1,099,510,579,200 vs DBN 80
```

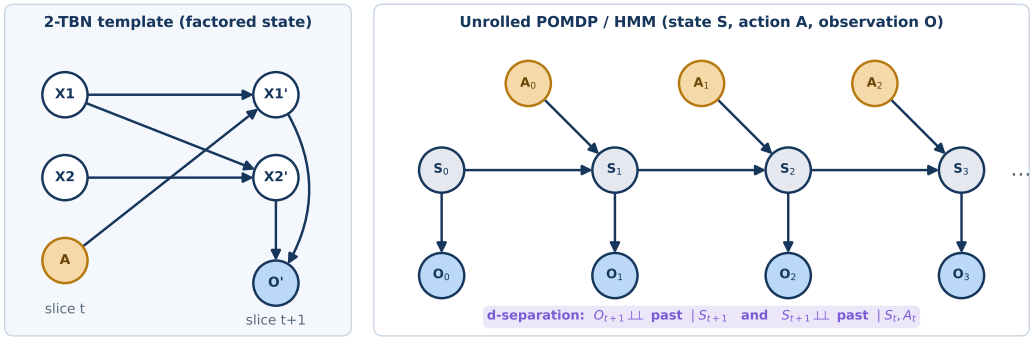


Figure 6.12. Dynamic Bayesian networks. Left: a 2-TBN with a factored state, an action node (amber) and an observation (blue). Right: the unrolled POMDP; the Markov properties of RL are d-separation statements in this graph.

LINK TO COURSE SESSION 17 · RL and MDPs

$P(s' \mid s, a)$ and $R(s, a)$ are CPDs of a DBN with the action as a parent of the next state; course session 21 adds the observation CPD, and updating the resulting *belief state* is BN inference (Session 7).

6.12.3 Multi-agent AI: distributed Bayesian networks

AI CONNECTION · Multi-Agent AI

In multi-agent sensing each agent observes a different part of the world. In a *multiply-sectioned* BN, each agent owns a sub-network, and neighbouring agents share *interface* variables (Fig. 6.13). If the interface between two agents d-separates their private variables, each agent can do inference locally and exchange only beliefs (messages) about interface variables, and the combined answer equals the centralized one — the multi-agent version of the Markov blanket.

Worked Example 6.19 (Analytical: what must the agents communicate?)**.** The illustrative network of Fig. 6.13 has the nine binary variables Wind, Cam(era), Fire, Terr(ain), Therm(al), Smoke, Alarm, Oper(ator) and Rep(ort), and the edges Wind→Cam, Fire→Cam, Fire→Therm, Terr→Therm, Fire→Smoke, Terr→Smoke, Smoke→Alarm, Smoke→Rep, Alarm→Oper. Agent 1 (drone) owns Wind, Cam, Fire; agent 2 (ground robot) owns Fire, Terr, Therm, Smoke; agent 3 (control room) owns Smoke, Alarm, Oper, Rep. Fire and Smoke are the interface variables. Is agent 1's private part $\{\text{Wind}, \text{Cam}\}$ d-separated from agent 3's private part $\{\text{Alarm}, \text{Oper}, \text{Rep}\}$ given $\{\text{Fire}\}$?

Solution. Every trail from Cam to the rest of the network leaves through Fire (Cam←Fire→Smoke or Cam←Fire→Therm); at Fire both edges point away from it, a fork, so observing Fire blocks the trail. Trails from Wind must pass Wind→Cam←Fire, where Cam is an unobserved collider without descendants, so they are blocked anyway. Hence $\{\text{Wind}, \text{Cam}\} \perp\!\!\!\perp_d \{\text{Alarm}, \text{Oper}, \text{Rep}\} \mid \{\text{Fire}\}$, and similarly agent 3's part is separated from agents 1–2 by $\{\text{Smoke}\}$. (every trail out of Alarm, Oper or Rep passes Smoke as a chain or fork node). Without evidence the parts are *not* separated (the trail Cam←Fire→Smoke→Alarm, with a fork at Fire and a chain at Smoke, is active). Agent 1 only needs to send its belief over the binary Fire (a two-number message), not its raw camera feed. Session 7 shows that these messages are precisely belief propagation.

PYTHON LAB · s06_examples.py · ex_multiagent()

```
{Wind,Cam} dsep agent-3 | {Fire}:  True; agent-3 dsep {Wind,Cam,Fire,Terr,Therm} | {Smoke}:
True
{Wind,Cam} dsep agent-3 | {}:  False
```

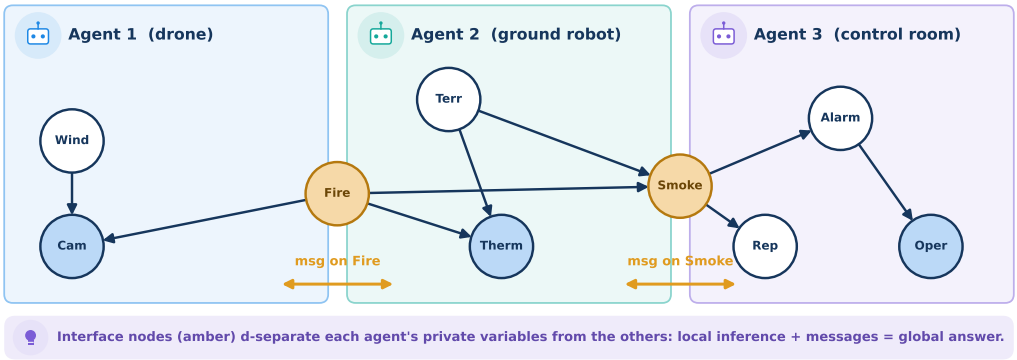


Figure 6.13. A multiply-sectioned BN for wildfire monitoring (illustrative). Interface nodes (amber) are shared between neighbouring agents and d-separate their private sub-networks.

Table 6.3. This session in the Advanced AI course: where the concepts reappear.

Concept from today	Reappears in	Role there
BN factorization	course 7, 10 (Sessions 7, 10)	efficient inference; decomposed likelihood
d-separation	course 12–14 (Sessions 12–14)	backdoor paths, collider bias, identification
Markov blanket	course 11 (Session 11), 27	Gibbs sampling; interfaces between agents
Markov equivalence	course 11, 13 (Sessions 11, 13)	CPDAGs from structure learning; limits of observational data
DBN / 2-TBN	course 17, 20, 21	transition models, factored MDPs, POMDP beliefs
Noisy-OR, neural CPDs	course 4, 21, 26	tool selection in agents; VAEs and autoregressive LLMs

LINK TO COURSE SESSION 27 · Multi-Agent AI

Interface variables are a shared language for coordination; CI of private signals given the true state justifies fusing them by multiplication, and its failure makes naive fusion overconfident.

6.12.4 Paradigms and deep generative models

LINK TO COURSE SESSION 2 · AI Paradigms

Deterministic CPDs make a BN a set of logical rules (symbolic), tabular or noisy-OR CPDs a statistical model, neural CPDs a neural model: the graph is neuro-symbolic glue.

LINK TO COURSE SESSION 4 · Generative Models

Deep generative models are BNs with neural CPDs. A VAE is the two-node network $Z \to X$ with $P(x \mid z)$ a neural network; an autoregressive model (including the next-token distribution of an LLM) is the *complete* DAG $x_1 \to x_2 \to \cdots$ in which x_i has all predecessors as parents — the chain rule of probability with no independence assumed, made tractable by sharing parameters rather than by dropping edges.

PART V · AI CONNECTIONS

6.13 Summary

THINK ABOUT IT **Two graphs encode the same independencies but disagree about the direction of an edge. Which observations, if any, could tell them apart?**

THINK ABOUT IT **Why are co-parents in the Markov blanket even though they are not adjacent to the node?**

THINK ABOUT IT **An LLM agent logs only the tool calls that succeeded. Which node is it conditioning on, and what does that do to the relations among the causes of success?**

Table 6.4. Cheat sheet of key formulas.

Object	Formula / rule
BN chain rule	$P(\mathbf{x}) = \prod_i P(x_i \mid \mathrm{pa}(x_i))$
Local Markov	$X_i \perp\!\!\!\perp \mathrm{nd}(X_i) \setminus \mathrm{pa}(X_i) \mid \mathrm{pa}(X_i) \;\Leftrightarrow$ factorization
Parameters	$\sum_i (k_i - 1) \prod_{j \in \mathrm{pa}(i)} k_j$
Noisy-OR	$P(Y = 0 \mid \mathbf{x}) = (1 - \lambda_0) \prod_i (1 - \lambda_i)^{x_i}$
d-separation	every path blocked: observed chain/fork node, or collider with no observed descendant
Markov blanket	$\mathrm{MB}(X) = \mathrm{pa} \cup \mathrm{ch} \cup \mathrm{pa}(\mathrm{ch})$; $P(x \mid \mathrm{rest}) \propto P(x \mid \mathrm{pa}) \prod_c P(c \mid \mathrm{pa}(c))$
Markov equivalence	same skeleton and same v-structures
Linear-Gaussian	$\Sigma = (I - B)^{-1} D (I - B)^{-\top}$; $\mathrm{Cov}(X_i, X_j) = \sum_k d_k \tau_{ik} \tau_{jk}$
CPT MLE	$\hat{\theta}_{x\mid u} = N(x,u)/N(u)$, s.e. $\approx \sqrt{\theta(1-\theta)/N(u)}$

Table 6.5. Common pitfalls.

Pitfall	Correct view
"Arrows mean causation"	a BN is a factorization; causal reading needs extra assumptions (Sessions 12–13)
Forgetting descendants	a collider is opened by observing any of its descendants
"Conditioning never hurts"	conditioning on a collider creates dependence (explaining away, Berkson)
Missing edge = no dependence	a missing edge X–Y asserts CI given *some* set (e.g. $\mathrm{pa}(Y)$ when $X \in \mathrm{nd}(Y)$), not marginal independence
Orienting every edge from data	only v-structures (and consequences) are identifiable: a CPDAG

Further reading. Koller & Friedman (2009), *Probabilistic Graphical Models*, Ch. 3 (representation, d-separation, I-maps, equivalence) and Ch. 5 (structured CPDs) and Ch. 6 (DBNs); Bishop (2006), *Pattern Recognition and Machine Learning*, Ch. 8.1–8.2; Murphy (2023), *Probabilistic Machine Learning: Advanced Topics*, chapter on directed graphical models; Pearl (2009), Ch. 1; Russell & Norvig (2021), *AIMA*, 4th ed., Ch. 13–14.

APPENDIX

6.14 Appendix: Proofs

Proof of Theorem 6.2 (soundness of d-separation)

Let X, Y, Z be disjoint node sets with $X \perp\!\!\!\perp_d Y \mid Z$, and let $A = X \cup Y \cup Z \cup \mathrm{an}(X \cup Y \cup Z)$; A is *ancestral* (it contains the parents of its members). Let M be the *moral graph* of the subgraph G_A: join every pair of parents of a common child in A, then drop all directions. Recall that a collider "or one of its descendants is in Z" iff the collider lies in $Z \cup \mathrm{an}(Z)$.

Lemma A (ancestral marginal). $P(\mathbf{x}_A) = \prod_{i \in A} P(x_i \mid \mathrm{pa}(x_i))$. *Proof.* No edge enters A from outside, so some topological order lists A first. Sum the joint (6.1) over the nodes outside A from the last one backwards: the last node appears only in its own CPD (its children, if any, come later and have already been summed out), which sums to one. What remains is the product over A.

Lemma B (graph separation $\Rightarrow$ CI). If $P(\mathbf{x}_A) = \prod_k \phi_k(\mathbf{x}_{C_k})$ where every C_k is a clique of an undirected graph H on A, and every path of H from X to Y meets Z, then $X \perp\!\!\!\perp Y \mid Z$. *Proof.* Let W be the nodes of $A \setminus Z$ joined to X by a path avoiding Z, and $R = A \setminus (W \cup Z)$; then $X \subseteq W$, $Y \subseteq R$. No clique meets both W and R (an edge between them would join that node of R to X), so each factor depends on $(\mathbf{x}_W, \mathbf{x}_Z)$ or on $(\mathbf{x}_R, \mathbf{x}_Z)$ only, and $P(\mathbf{x}_A) = g(\mathbf{x}_W, \mathbf{x}_Z)\, h(\mathbf{x}_R, \mathbf{x}_Z)$. By Session 5's factorized form of CI, $W \perp\!\!\!\perp R \mid Z$; decomposition gives $X \perp\!\!\!\perp Y \mid Z$.

Lemma C (walks). Call a walk (a trail in which nodes may repeat) *active* given Z if each of its non-collider occurrences is outside Z and each collider occurrence lies in $Z \cup \mathrm{an}(Z)$. If an active walk from X to Y exists, an active trail exists. *Proof.* Take an active walk with fewest edges and suppose a node v occurs twice; delete the loop between the two occurrences. In the shorter walk, v keeps the incoming side of its first occurrence and the outgoing side of its second. If $v \in Z$, both original occurrences were colliders, so the new one is a collider in Z: allowed. If $v \notin Z$ and the new occurrence is a non-collider: allowed. If $v \notin Z$ and it is a collider, then either one original occurrence was already a collider (so $v \in \mathrm{an}(Z) \cup Z$), or the deleted loop leaves v along an outgoing edge; following the loop, it must turn upwards at some collider w before returning to v (otherwise a directed cycle), w is a descendant of v and $w \in Z \cup \mathrm{an}(Z)$, hence $v \in \mathrm{an}(Z)$: allowed. The shorter walk is active, a contradiction; so the shortest active walk is a trail.

Lemma D (d-separation $\Rightarrow$ moral separation). Every path of M from X to Y meets Z. *Proof.* Suppose π is a path of M from $x \in X$ to $y \in Y$ avoiding Z. Replace each moral edge $u - w$ that is not an edge of G by $u \to c \leftarrow w$ through a common child $c \in A$. The resulting walk has all non-colliders on π, hence outside Z, and all colliders in A. A collider in $Z \cup \mathrm{an}(Z)$ is fine; call the others *bad* — each is an ancestor of X or of Y but not of Z. If some bad collider is an ancestor of a node $x' \in X$, take the last such one, c, along the walk, and replace the part before c by the reversed directed path $x' \leftarrow \cdots \leftarrow c$; its nodes avoid Z (else $c \in \mathrm{an}(Z)$) and are chain nodes, and c itself becomes a chain node. Symmetrically, take the first remaining bad collider c' that is an ancestor of some $y' \in Y$ and replace the part after c' by $c' \to \cdots \to y'$. No bad collider is left, so the walk is active, and by Lemma C there is an active trail from X to Y — contradicting $X \perp\!\!\!\perp_d Y \mid Z$.

Conclusion. By Lemma A, $P(\mathbf{x}_A)$ is a product of CPDs, and the scope $\{X_i\} \cup \mathrm{pa}(X_i)$ of each is a clique of M (the parents were married and each is joined to the child). By Lemma D, Z separates X from Y in M; by Lemma B, $X \perp\!\!\!\perp Y \mid Z$ in the marginal on A, hence in P. □

This "moralized ancestral graph" construction reappears from the opposite direction in Session 8, when Bayesian networks are converted into Markov networks.

CONNECTIONS AND TAKEAWAYS

We now have a representation (the DAG) that compactly encodes a joint distribution's factorization, and an algorithm (d-separation, implemented in linear time by Bayes ball) that reads off every conditional independence the graph implies — without ever touching the numbers in Θ. This closes the loop opened in Session 5: independence assumptions are no longer something we verify laboriously by algebra, but something we can see.

But notice what we have *not* yet done efficiently: every posterior in this session, like $P(I = \text{high} \mid L = \text{weak})$, was computed by brute-force enumeration of the full joint table, which is exactly what we showed to be intractable in Session 5. Knowing the graph's independence structure is necessary for efficient computation, but it is not the computation itself. **Session 7** turns this structural knowledge into algorithms — variable elimination, belief propagation and MAP inference — that compute exact answers to probability queries by systematically exploiting exactly the structure d-separation reveals, without ever building the full joint table.